



INAOE

**Instituto Nacional de Astrofísica,
Óptica y Electrónica.**

REPORTE TECNICO

No. 665

**Implementación de un
“Hardware In the Loop” para un
Simulador de CubeSats**

Ing. Said López Escobedo*
Dr. Manuel Sanjurjo Rivo**
Dr. Rogerio Enríquez Caldera***

* Posgrado en Ciencia y Tecnología del Espacio

** Universidad Carlos III en Madrid, Departamento de
Bioingeniería e Ingeniería Aeroespacial.

*** INAOE/CRECTEALC

COORDINACION DE ELECTRONICA

©INAOE 2019

Derechos Reservados

El autor otorga al INAOE el permiso de reproducir y distribuir copias de este reporte técnico en su totalidad o en partes mencionando la fuente.





Instituto Nacional
de Astrofísica,
Óptica y Electrónica



Universidad
Carlos III de Madrid

Maestría en Ciencia y Tecnología del Espacio

Implementación de un “Hardware In the Loop” para un Simulador de CubeSats

Ing. Said López Escobedo

Dr. Manuel Sanjurjo Rivo

Dr. Rogerio Enríquez Caldera

Keywords: Hardware in the Loop (HIL), Simuladores de Satélites, Control de Orientación.

Octubre 2019

Contenido

1. Introducción	2
1.1. Objetivos	2
1.2. Descripción general del sistema	3
1.3. Simulador	4
1.4. Hardware In the loop	4
2. Hexápodo	4
2.1. Características generales	5
2.2. Funcionamiento	6
2.3. Configuración inicial	6
3. Simulador	9
3.1. GUI del Simulador: Configuración inicial	10
3.2. GUI del Simulador: Visualización y Gráficas	13
4. Hardware In the Loop	16
4.1. Hardware Arduino	16
4.2. Sensores	20
4.2.1. Sistema de navegación inercial	20
4.2.2. GY 85	21
4.2.3. Estimación de la orientación	22
5. Implementación general del HIL Simulator	23
5.1. Módulo Hexápodo	24
5.2. Giroscopio Virtual	24
5.3. IMU Real	25
6. Resultados	26
6.1. Verificación de funcionamiento del hexápodo	26
6.2. Instalación y operación del IMU	27
7. Conclusiones y recomendaciones	37
8. Referencias	38

1. Introducción

El denominado “Hardware In the Loop” (HIL) es una técnica que usa señales reales de un microcontrolador y son conectadas a un sistema de pruebas que simula la realidad, engañando así al microcontrolador para que este piense que está en un sistema real. La parte del simulador incluye toda la complejidad del sistema real mediante modelos matemáticos de todos los estados relacionados con el sistema.

Como un ejemplo de HIL [1], supóngase el módulo de control del motor de un automóvil responsable de convertir las medidas de los sensores de comando en acción reales (v.g. controlar la relación entre aire y combustible).

Una prueba en HIL reemplaza el motor con una simulación que consta de hardware y software que interactúan con las entradas y salidas reales como si el motor físico estuviera presente (Ver Fig. 1). Debido a que los cambios se pueden hacer en software, se puede incorporar los cambios de variables del motor o el módulo de control del motor y probar una gran variedad de escenarios como sea necesario.

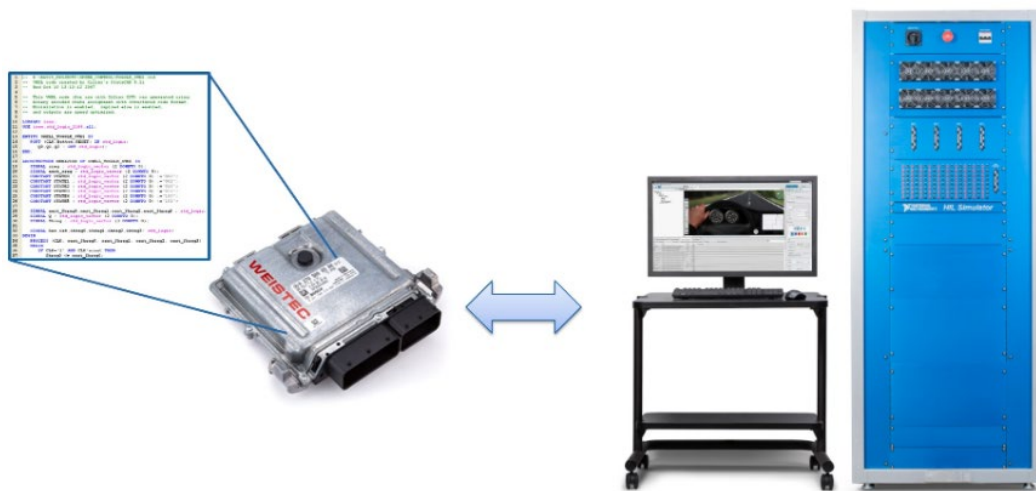


Fig. 1 Motor Reemplazado con Simulación HIL [1]

Lo anterior permite realizar pruebas sin temor y sin el riesgo de utilizar un sistema físico y costoso. Además, en muchos casos, el sistema real es más costoso que un simulador fiable en tiempo real y por ello es más recomendable su simulación, por lo que es más económico el desarrollo y prueba de un sistema simulador con HIL que al sistema real.

1.1 Objetivos

Objetivo General

Generar una interfaz de usuario para una plataforma inercial de seis grados de libertad y ser utilizada en pruebas de orientación de pequeños satélites.

Objetivos específicos

Preparar el software y el hardware de la plataforma para emular, con las restricciones propias de la plataforma, los movimientos de orientación para trayectorias en el espacio:

1. Probar y operar debidamente la plataforma inercial.

2. Programar los movimientos de la plataforma para emular orientación restringida en asociación con simulador de orientación de pequeños satélites en trayectos en ambiente espacial.
3. Probar, instalar y operar los sensores inerciales en la plataforma.
4. Implementar, conjuntamente con el desarrollo anterior, el HIL para un simulador de Cubsats.
5. Analizar y comparar resultados obtenidos con usando el HIL contra los obtenidos por los instrumentos virtuales del simulador.

1.2 Descripción general del sistema

El diagrama de la Fig. 2 presenta de forma general el sistema descrito en este documento compuesto por un HIL para un simulador de navegación y rotación para pequeños satélites tipo CubeSat realizado en software.

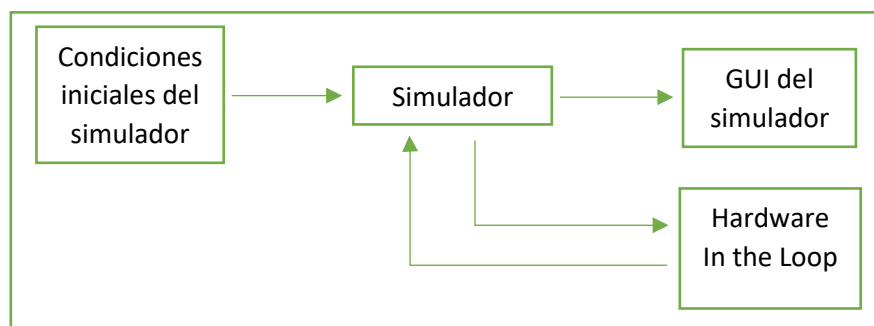


Fig. 2 Diagrama de la descripción general del sistema HIL

El simulador específico que se usará en este proyecto fue realizado por fabricación propia en INAOE, México denominado SatSim4Cubes [2]. En dicho simulador, las condiciones iniciales de trayectorias y rotaciones del satélite son introducidas por medio de su GUI ("Graphical User Interface") y que además esta GUI permite la visualización y apreciación gráfica de los movimientos solicitados. El HIL se conforma al añadirle a este simulador una plataforma de 6 grados de libertad e instrumentos extras de medición de movimiento pertenecientes al Departamento de Bioingeniería e Ingeniería Aeroespacial de la Universidad Carlos III de Madrid.

Considerando lo anterior, y basándose en esa misma figura, se aprecia de manera integral lo que ahora será denominado **HIL Simulator** y que está compuesto por cuatro elementos:

1. **Condiciones iniciales del simulador.** Son las configuraciones iniciales que están basadas en los parámetros orbitales, características principales del Cubesat, y los coeficientes del sistema de control. Estos datos son fundamentales para que el simulador inicie correctamente su funcionamiento.
2. **Simulador.** Pretende reproducir un entorno real basado en modelos matemáticos del ambiente espacial, de los sensores y actuadores (instrumentos virtuales) que incluyen las ecuaciones dinámicas y cinemáticas de un satélite, y de donde se obtendrá la toma de datos y visualización de la trayectoria y rotación del Cubesat bajo estudio.

3. **GUI del simulador.** Es la interfaz gráfica de usuario en donde de manera visual se muestran los resultados arrojados por el simulador tanto en forma de animación como de diversas gráficas de los datos de interés.
4. **Hardware In the Loop.** Está integrado por el Hexápodo, un microcontrolador y los sensores que son interconectados al simulador para que posteriormente exista una interacción entre los instrumentos virtuales, el Hexápodo y los sensores reales adicionados al Hexápodo. Así, se constituye un sistema completo que permite visualizar por medio de la GUI del simulador de software un auténtico y completo HIL Simulator.

Posteriormente se presentarán con mayor detalle estas cuatro partes para diseñar, implementar y por ende, comprender el HIL Simulator.

1.3 Simulador

Para el software de la sección de simulación es muy común, hoy en día, utilizar Matlab junto con la integración de Simulink dada su flexibilidad, modularidad y costo. Además, este software permite implementar diseños basados en modelos y generar automáticamente códigos para realizar pruebas y verificar sistemas HIL.

El software de simulación debe combinarse con el hardware que no solamente responda a las especificaciones de hardware (tipo de conectores y entradas/salidas, etc.) sino que también permita la introducción de fallas y la habilidad de programar escenarios realistas de prueba.

La calidad del software de simulación es de suma importancia [1] ya que aunado al HIL, se pueden hacer pruebas que se consideren de valor.

1.4 Hardware In the Loop

En este reporte particular se describirá primeramente el HIL desarrollado para CubeSats a través de un hexápodo que hará la función de simular los movimientos de un Cubesat.

2. Hexápodo

El Hexápodo como se muestra en la Fig. 3 es una plataforma robótica de movimiento de seis grados de libertad que se puede usar en una gran variedad de áreas de investigación y educativas. El Hexápodo es accionado por motores de corriente continua que hacen que la plataforma sea suficientemente precisa, de bajo mantenimiento y de alto rendimiento, tiene incorporado codificadores ópticos de alta resolución que permiten medir la posición angular de cada motor por eje grado de libertad [3].

Para la seguridad del Hexápodo, se emplea un control de freno en cada motor cuando las juntas alcanzan su límite para que éstos no se dañen. Los seis brazos de la plataforma se encuentran en una base rectangular plana.

El Hexápodo se controla a través de QUARC® y MATLAB® / Simulink®. Para ello se explicará en secciones posteriores como es la interacción y las configuraciones iniciales que se deben de realizar para un correcto funcionamiento del Hexápodo durante la simulación.



Fig.3 Plataforma de movimiento 6 DOF

2.1 Características generales

Es importante conocer las características y parámetros principales del Hexápodo ya que nos ayudará a identificar y conocer los límites de funcionamiento cuando éste esté en operación. Algunas de las características principales que describen al Hexápodo son las siguientes [4]:

- Amplificador incorporado de alto rendimiento
- Mecanismo de husillo de bolas de alta precisión
- Software de vigilancia de seguridad incorporado
- Interfaz fácil a través de conexión USB universal
- Componentes mecanizados precisos, rígidos y resistentes
- Modelos y parámetros del sistema completamente documentados.
- Fácil integración de estructuras, sensores y actuadores de terceros.
- Circuito lógico de freno de seguridad y frenos mecánicos incorporados.
- Codificadores ópticos de alta resolución para medir los ángulos de las articulaciones.
- Diseño de operación y control altamente flexible con MATLAB /Simulink a través de QUARC

A continuación, se presenta en la Tabla los principales parámetros del Hexápodo.

Descripción	Valor
Dimensiones	1.1 m x 1.1 m x 0.75 m
Masa	100 kg
Radio de la plataforma	0.25 m
Largo del brazo	0.375 m
Carga útil máxima	100 kg
Espacio de trabajo	± 13 cm (x), ± 7.5 cm (y), ± 7.5 cm (z) $\pm 10^\circ$ (roll) $\pm 11.5^\circ$ (pitch), $\pm 13.5^\circ$ (yaw)
Velocidad máxima	0.67 m/s (x), 0.67 m/s (y), 0.67 m/s (z) 152.7°/s (roll) 152.7°/s (pitch), 80.6°/s (yaw)
Aceleración máxima	1 g
Ancho de banda operacional	0-10 Hz
Actuador fuerza máxima	403 N
Precisión de posicionamiento	0.06mm
Recorrido del actuador	0.3 cm
Límite de corriente pico	4.7A
Resolución del codificador	10,000 cont / rev

Tabla.1 Parámetros del sistema (Hexápodo)

2.2 Funcionamiento

Se describe cómo conectar el Hexápodo para un correcto funcionamiento.

Antes de hacer uso del Hexápodo es importante mencionar que se tiene que tener actualizado MATLAB, así como la licencia de QUARC, ya que si no se tiene actualizada la licencia QUARC no permitirá MATLAB hacer uso de los modelos de Simulink para el Hexápodo. En la Fig. 4 se muestra la licencia QUARC que se utilizará.

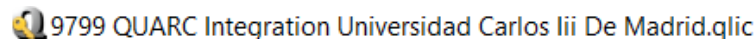


Fig. 4 Licencia QUARC

Para configurar el Hexápodo, es conveniente seguir los siguientes pasos [5] esquematizados en la Fig. 5:

1. Conecte el interruptor de parada de emergencia al conector de parada de emergencia en el Hexápodo.
2. Conecte el cable USB que se ubica a un costado del Hexápodo a un puerto USB de su PC de escritorio o portátil.
3. Conecte el cable de alimentación del Hexápodo a una fuente de alimentación de 100-240V.
4. Encienda el interruptor que está a un costado del Hexápodo.
5. El controlador del Hexápodo debería instalarse automáticamente. De lo contrario, es posible que no haya instalado todo el software necesario para admitir el dispositivo.

Nota: El Hexápodo no se puede operar sin conectar el interruptor de parada de emergencia.

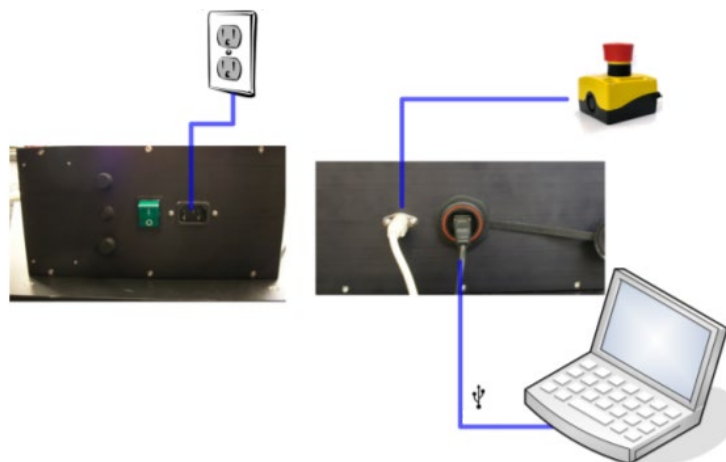


Fig.5 Conexiones del Hexápodo

2.3 Configuración inicial

Cuando el Hexápodo se enciende, las posiciones medidas por los codificadores de todas las uniones se restablecen a 0, independientemente de dónde se encuentre la unión de la

base del Hexápodo por lo que es necesario realizar una calibración antes de poder utilizar el Hexápodo [6].

En este apartado se explica cómo se usan los modelos de Simulink proporcionados por el software QUARC para ejecutar lo siguiente [6]:

1. Calibración: Se ejecuta una rutina automática y restablece los codificadores a cero. Es necesario una vez encendido el Hexápodo.
2. Inicio: Lleva al hexápodo a una posición “Inicio”, donde cada articulación se encuentra en una posición media.

Calibración:

El modelo de Simulink “Hexapod_Calibrate.mdl” que se muestra en la Fig. 6, coloca cada una de las seis articulaciones a lo largo de la guía lineal hasta que alcanzan el interruptor de límite. Una vez que las seis articulaciones se han colocado en las posiciones del interruptor de límite, los codificadores se reinician y el modelo se detiene. Si las uniones ya están en la posición calibrada, entonces el controlador solo reiniciará los codificadores y lo detendrá.

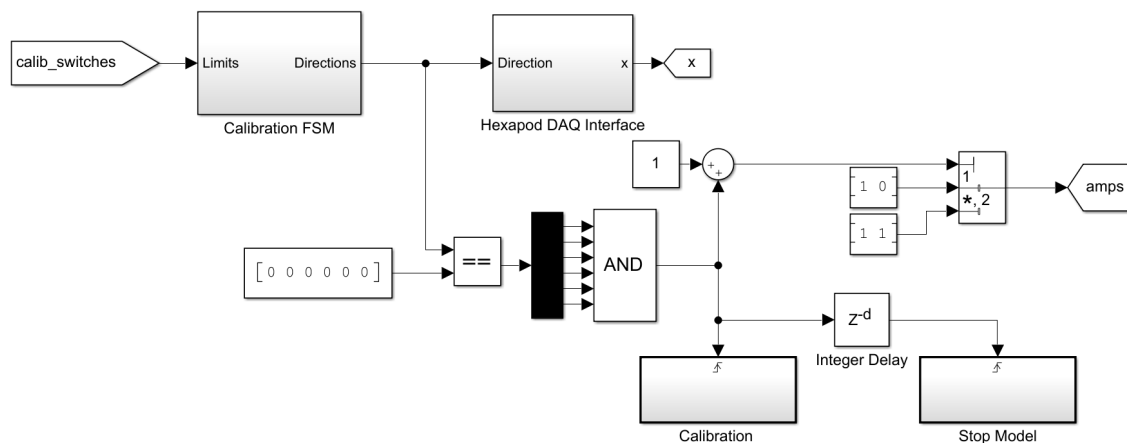


Fig. 6 Hexapod_Calibrate.mdl

Siga estos pasos para calibrar el sistema:

1. Asegúrese de que el botón rojo del interruptor de parada de emergencia esté en la posición “arriba” para habilitar el amplificador. El amplificador no se puede habilitar cuando el botón está en la posición “abajo” por lo que se debe girar el botón para liberarlo y que cambie a “arriba” lo cual asegura que está en posición habilitada.
2. Ejecute “setup_hexapod.m” en MATLAB de la Fig. 7.
3. Abra el modelo Simulink llamado “Hexapod_Calibrate.mdl” (Fig. 6).
4. Haga clic en Build model para generar el controlador.
5. Encienda el Hexápodo colocando el interruptor de encendido en la posición ON. Escuchará un pequeño chasquido al momento de encenderlo.
6. Ejecute el modelo de Simulink dando “clic” en Connect To Target y posteriormente en Start.
7. Presione el botón BY PASS que se encuentra en el interruptor de parada de emergencia, escuchara un chasquido y a continuación cada una de las articulaciones del Hexápodo comenzara a moverse hasta que llegue a los límites por lo que se activaran los interruptores de límite.

8. El controlador se detiene automáticamente cuando cada una de las seis articulaciones ha alcanzado su interruptor de límite.
9. Escuchara un chasquido indicando que ha terminado de ejecutarse el modelo de Simulink.

Nota de precaución: PRESIONE HACIA ABAJO el BOTÓN ROJO del interruptor de parada de emergencia para detener el Hexápodo. Esto desactiva el amplificador y corta la potencia del motor de corriente directa.

```
clear all;
% Coupling Circumference
Circ = 10e-3;
% Radius
r = Circ/2/pi;
K_ENC = 2*pi/10000;
% Encoder / Motor Channels
Channels = [0:5];
% Motor Torque Constant
Kt = 19.3*0.0254*0.2780139;

% Dynamic Collision Avoidance
q_upper=310e-3;
q_lower=30e-3;
min_separation=170e-3;
```

Fig. 7 setup_hexapod.m

Una vez calibrado el sistema, se procede a ejecutar el siguiente programa “Inicio”.

Inicio:

Antes de ejecutar cualquier modelo de Simulink, el Hexápodo debe estar en la posición INICIO. Esto se define cuando cada joint está en posición centrada.

El modelo Hexapod_Home.mdl de Simulink, que se muestra en la Fig. 8, se ejecuta para centrar todas las uniones lineales en la base.

Siga estos pasos para iniciar el sistema:

1. Asegúrese de que el botón rojo del interruptor de parada de emergencia esté en la posición “arriba” para habilitar el amplificador. Recuerde que el amplificador no se puede habilitar cuando el botón está en la posición “abajo” por lo que gire el botón para asegurarse de que esté en la posición habilitada.
2. Asegúrese de haber calibrado el hexápodo primero.
3. Ejecute “setup_hexapod.m” en MATLAB (Ver Fig. 7).
4. Abra el modelo Simulink llamado Hexapod_Home.mdl, (Ver Fig. 8).
5. Haga clic en Build model para generar un controlador.
6. Ejecute el modelo de Simulink dando clic en Connect To Target y posteriormente en Start.
7. Presione el botón BY PASS que se encuentra en el interruptor de parada de emergencia, escuchara un chasquido y a continuación cada una de las articulaciones del Hexápodo

comenzara a moverse hasta que llegue a los limites por lo que se activaran los interruptores de límite.

8. El controlador se detiene automáticamente cuando cada una de las seis articulaciones ha alcanzado su interruptor de límite.
9. Escuchara un chasquido indicando que ha terminado de ejecutarse el modelo de Simulink.

Nota de precaución: PRESIONE HACIA ABAJO el BOTÓN ROJO del interruptor de parada de emergencia para detener el Hexápodo. Esto desactiva el amplificador y corta la potencia del motor de corriente directa.

Asegúrese de que el Hexápodo esté en estatus de “Inicio” antes de ejecutar cualquier otro programa.

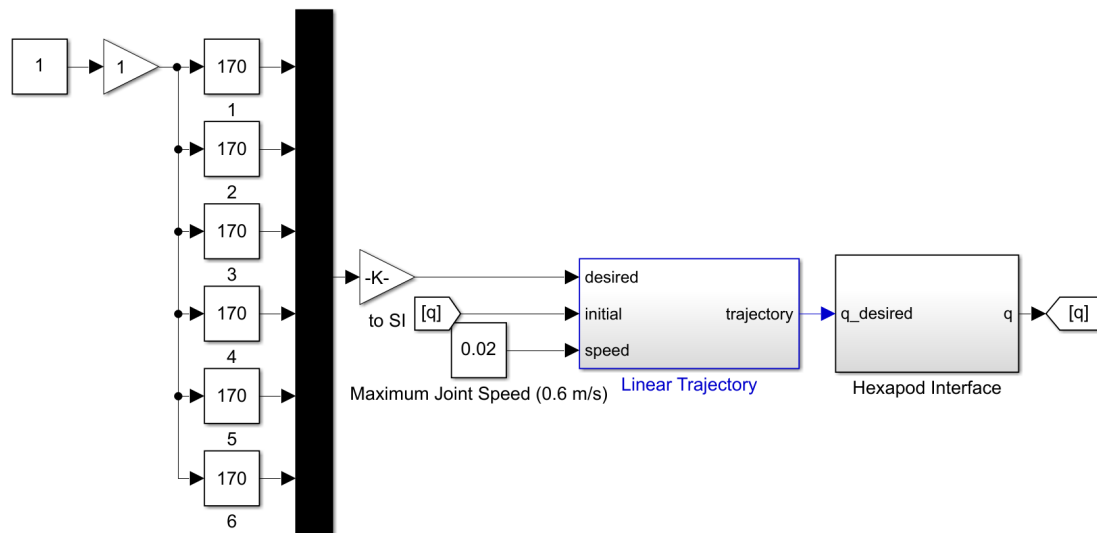


Fig. 8 Hexapod_Home.mdl

3. Simulador

Como se mencionó anteriormente, en muchos casos un sistema real es más costoso que un simulador en tiempo real, por ello es más recomendable una simulación con un HIL. Así pues, ahora la interfaz de usuario del simulador debe ser complementada para permitir realizar pruebas con el HIL. En este reporte se emplea la plataforma inercial de seis grados de libertad mencionada con anterioridad para realizar pruebas de orientación en pequeños satélites llamados CubeSat's.

Al crear la interfaz para el HIL se podrá analizar y comparar resultados obtenidos por la plataforma contra los obtenidos por los instrumentos virtuales del simulador, de forma tal que pueda visualizar los resultados además de poder manipular variables del sistema, de una manera flexible sin entrar en detalles de la programación ni en fórmulas matemáticas complejas que corresponderían al modelo de un Cubesat bajo estudio.

3.1 GUI del simulador: Configuración Inicial

La interfaz de usuario de este reporte forma parte de un simulador que se realizó como trabajo final de Maestría de Said en INAOE [2] y que está basado principalmente en el modelado de orientación de un CubeSat's en una órbita definida por las configuraciones disponibles en el simulador además de las características principales del Cubesat, así como su sistema de control sencillo tipo PID (aunque después pueden implementarse sistemas de control más sofisticados y modernos tan solo cambiando el SCRIP correspondiente en el SatSim4Cubes). El Hexápodo hará la función de simular los movimientos del Cubesat. Por tanto, primero describiremos el GUI del SatSim4Cubes.

Al ejecutar por primera vez el simulador se muestra una ventana de configuración como se muestra en la Fig. 9. SatSim4Cubes nos brinda 4 secciones en donde el usuario tiene que introducir los parámetros (mismos que posteriormente serán utilizados por un modelo en Simulink en donde se integrara con el Hexápodo):

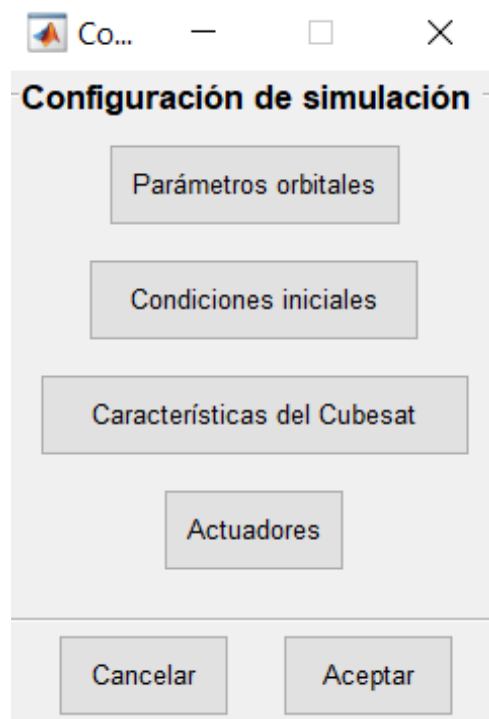


Fig. 9 Configuración inicial de simulación

- i) Parámetros Orbitales (Fig. 10). La representación gráfica de la fig. 10 es para que el usuario pueda visualizar la ubicación de la órbita, así como la rotación de la Tierra correspondiente al día bajo estudio. Aquí es donde se introducirán los datos de la órbita a simular tales como: Fecha actual, Hora Universal, así como el número de orbitas a simular; posteriormente, se introducirán los 6 parámetros orbitales que definirán a la órbita (a la derecha de la Fig. 10 el mundo con la órbita se modificara automáticamente cada que el usuario modifique un parámetro).

Parámetros orbitales

Fecha actual: 2019 June 11

Hora universal: 9 : 0 : 0

Numero de orbitas a simular: 4

Semi eje mayor (Km): 6950

Excentricidad: 0

Anomalía media (deg): 0

Inclinación (deg): 42

A.R del Nodo Asc (deg): 16

Argumento perigéo (deg): 0

Periodo orbital: 96.1025

Cancelar Aceptar

Fig. 10 Parámetros orbitales

- ii) Condiciones iniciales (Fig. 11). Se refiere a la orientación inicial del Cubesat con respecto al sistema de referencia orbital, así como su velocidad angular inicial del Cubesat en ese mismo sistema de referencia, por tanto, para proporcionar estas condiciones iniciales no se debe considerar la velocidad angular orbital a la que se mueve el satélite en una órbita dada. Acorde a la Fig. 11, se puede introducir la posición de inicio tanto en forma de ángulos de Euler o en forma de Cuaternion. Se recomienda que la posición y velocidad iniciales siempre sean de valor 0 ya que así podremos verificar cuando el sistema este de forma estable y no se origine algún movimiento provocado por la acción de las ruedas inerciales simuladas.

Condiciones iniciales

Posición inicial

☒ Euler ☐ Cuaternion

Orden de rotación: ZYX

φ: 0 deg ☒ Deg ☐ Rad

θ: -0 deg

ψ: 0 deg

Velocidad angular inicial

ωx: 0 deg/s ωy: 0 deg/s ωz: 0 deg/s

Cancelar Aceptar

Fig. 11 Condiciones iniciales

- iii) Características del Cubesat (Fig. 12). Este apartado se refiere a las dimensiones, masa, centro de gravedad y la matriz de inercia correspondiente. El usuario tiene la libertad de escoger estas dimensiones del Cubesat (los valores por omisión los proporciona un Cubesat de 1 unidad - 1U con un tamaño de 10x10x10 cm - el default para la masa es de 1.2 Kg – dentro de los 1.33 Kg para 1U – mientras que el centro de gravedad se considera que siempre estará en la posición central del Cubesat).

Características del Cubesat

Dimensiones CubeSat

a: 0.1 m b: 0.1 m c: 0.1 m

Masa

m: 1.2 kg

Centro de gravedad

x y z

c.g: 0 0 0 m

Matriz de inercia

	X	Y	Z
X	0.002	0	0
Y	0	0.002	0
Z	0	0	0.002

kg*m² ☐ Activar

Cancelar Aceptar

Fig. 12 Características del Cubesat

iv) Actuadores (Fig 13). La última sección se refiere al actuador que utilizara el Cubesat para realizar el control de la orientación, en este caso los parámetros que se observan en la Fig. 13 corresponden a una rueda inercial comercial específica, por lo que es recomendable que no modifique esta sección a menos que tenga todos los datos necesarios de la rueda inercial comercial, ya que si introduce datos que no corresponden a una rueda inercial conocida, el control no se comportara de manera correcta y por lo tanto la simulación tendrá errores numéricos que no corresponderán a una simulación real. Por lo pronto, únicamente se están considerando actuadores tipo ruedas inerciales dejando a futuro los actuadores tipo torca magnética.

Fig. 13 Actuadores

De manera resumida se muestra los pasos que se requieren para que pueda funcionar el simulador de manera correcta:

1. Ejecute "Labcube.m" en MATLAB.
2. Se abrirá una ventana de configuración de simulación.
3. Haga clic en los parámetros orbitales y configure los parámetros de la órbita.
4. Haga clic en las condiciones iniciales y configure la posición y velocidades iniciales.
5. Haga clic en las características del Cubesat y configure las dimensiones y masa.
6. Haga clic en los actuadores y configure las ruedas inerciales que utilizara.
7. Haga clic en Aceptar para iniciar la simulación.

Nota: Asegúrese de que el Hexápodo sea calibrado y esté en la posición de inicio antes de ejecutar el simulador.

3.2 GUI del simulador Visualización y Graficas

La interfaz gráfica de salida es de gran apoyo cuantitativo y cualitativo ya que nos permite visualizar la trayectoria y la orientación del satélite mediante una animación en el tiempo de simulación de interés, así como mostrar las gráficas de los datos de los sensores y actuadores virtuales. Posteriormente, esta sección permitirá igualmente apreciar los gráficos de los sensores reales que se utilizarán en HIL, es decir, la visualización y las gráficas corresponden ya propiamente al HIL.

Una vez terminada la configuración de la simulación de la sección anterior y dar pulsar el botón “aceptar”, se abrirán nuevas ventanas como se muestra en la Fig. 14 en donde podremos observar, una animación del satélite ejecutando la órbita introducida con anterioridad. Debajo de la animación, también se observa una subventana de comandos que nos ayudará a definir la orientación del Cubesat deseada por el usuario; junto a esas ventanas y para el caso de la interfaz HIL, presentará dos opciones para elegir ya sea los sensores virtuales o los sensores reales.

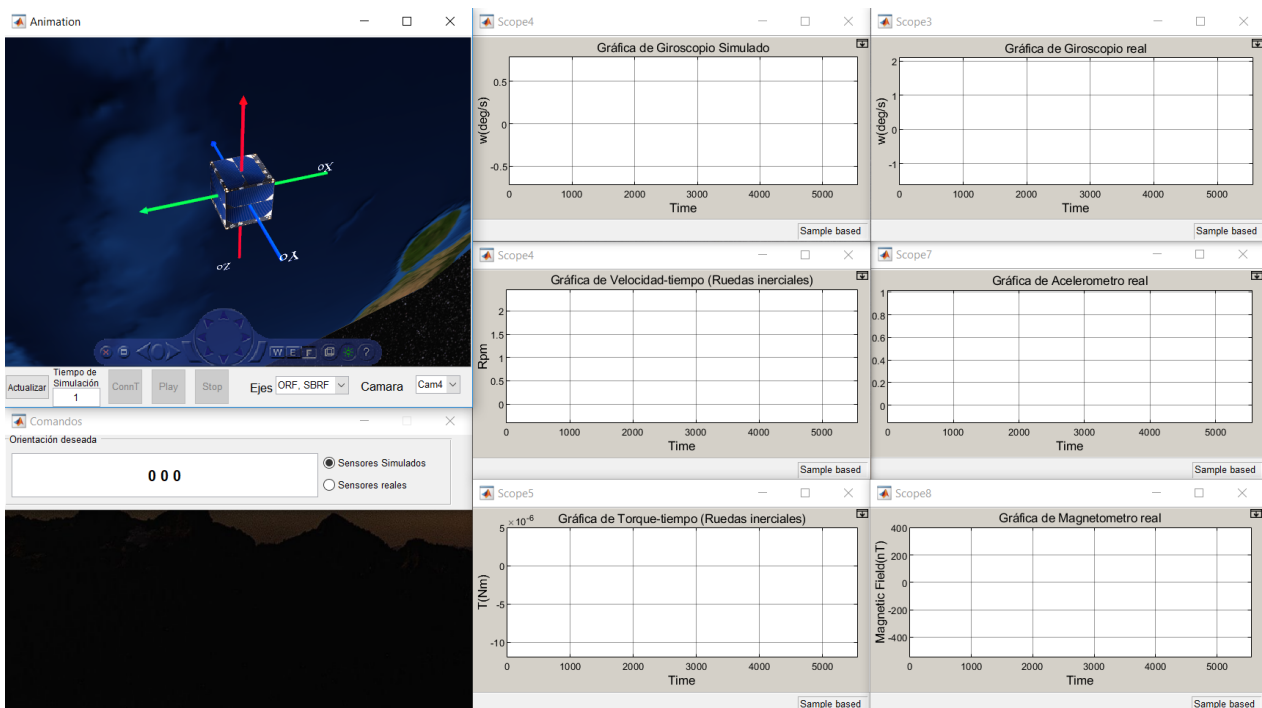


Fig.14 GUI del simulador

La ventana de comandos de orientación (Ver Fig. 15) nos ayudara a definir la triada x y z de orientación deseada por el usuario quien debe introducir los valores de los ángulo en grados comenzando por el movimiento “Roll” que correspondería al eje X, luego el movimiento “Pitch” que correspondería al eje Y, y finalmente el movimiento “Yaw” que correspondería al eje Z.

Ya en el caso de la interfaz HIL, las dos opciones que aparecen a un lado de la orientación deseada es para elegir los sensores virtuales o los sensores reales. Se recomienda iniciar con los sensores virtuales para evitar cualquier factor de ruido asociado a los sensores reales con el correspondiente efecto indeseado en el movimiento del Hexápodo. Esto último, se explicará en secciones posteriores.

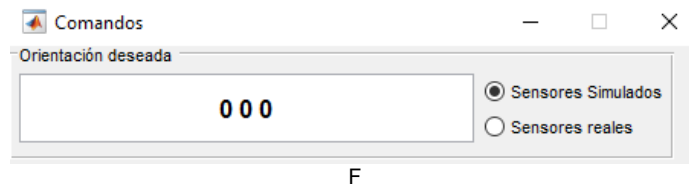


Fig. 15 Ventana de comandos

En cuanto a las gráficas de datos, se muestran 6 graficas en donde podremos observar los sensores (virtuales y reales), así como los datos de los actuadores que permiten cambiar la orientación del Cubesat y que son representados por la velocidad de las ruedas inerciales simuladas correspondientemente con el torque que se genera.

Ahora bien, específicamente, en la ventana de animación (Ver Fig. 16), se podrá observar el satélite que está orbitando alrededor de la Tierra conforme a los parámetros orbitales que se introdujeron en la configuración del simulador y, se podrán observar los ejes de referencia de la órbita y el satélite. Para que la animación se muestre de manera correcta, por recomendación el tiempo de simulación que se trabaja es 1 seg.

Los puntos de vista de la animación se podrán cambiar seleccionando entre 5 posibles cámaras ya pre-establecidas en la misma ventana.

En este caso para la interfaz de HIL, se tienen presentes tres botones para la simulación, el primero de ellos es "ConnT" que representa el conectar el Target del Hexápodo al simulador, posteriormente se habilita el botón "Play" para empezar la simulación y el botón "Stop" para para la simulación.

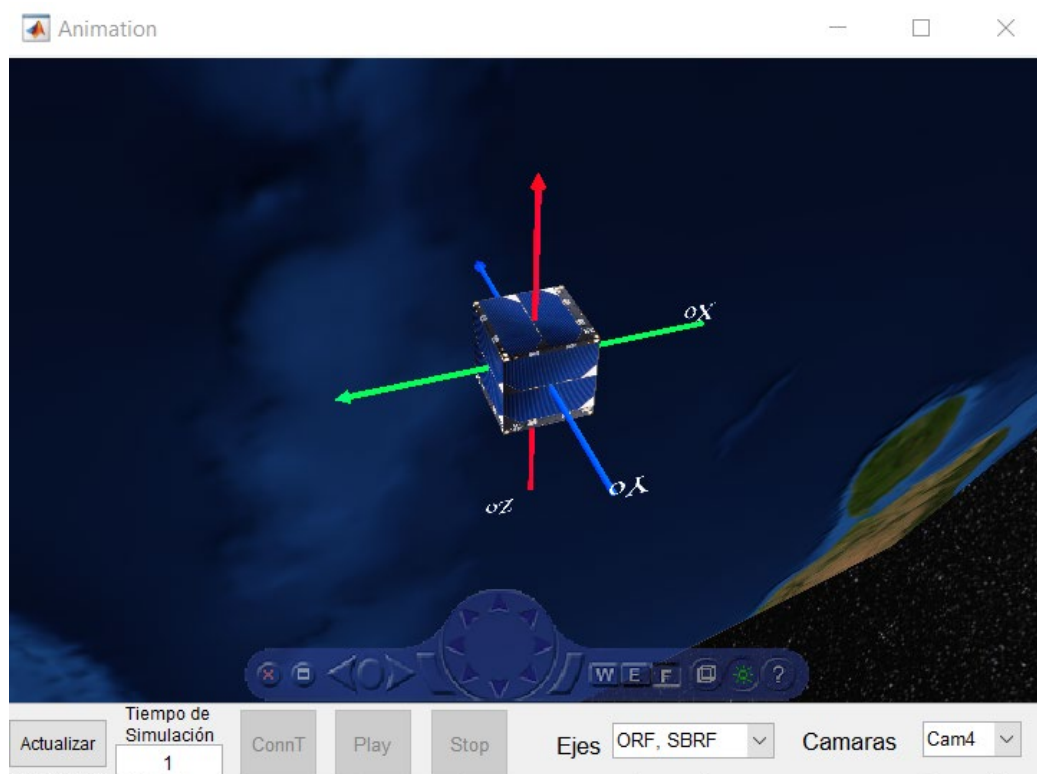


Fig. 16 Animación Cubesat

En la Fig. 17, se detallan las 6 graficas de datos tanto del simulador *per se* (que por el momento solo es una que corresponde al giroscopio virtual) como de las tres gráficas propias de la interfaz de HIL en donde podremos observar los sensores reales en este caso el

giroscopio, acelerómetro y magnetómetro. Las dos graficas finales presentan las velocidades de las ruedas inerciales simuladas, junto con los torques que se generan para cambiar la orientación del Cubesat.

De estas gráficas, y dado que por el momento solo tenemos giroscopio virtual, centraremos nuestro estudio en las gráficas del giroscopio real y el giroscopio simulado.

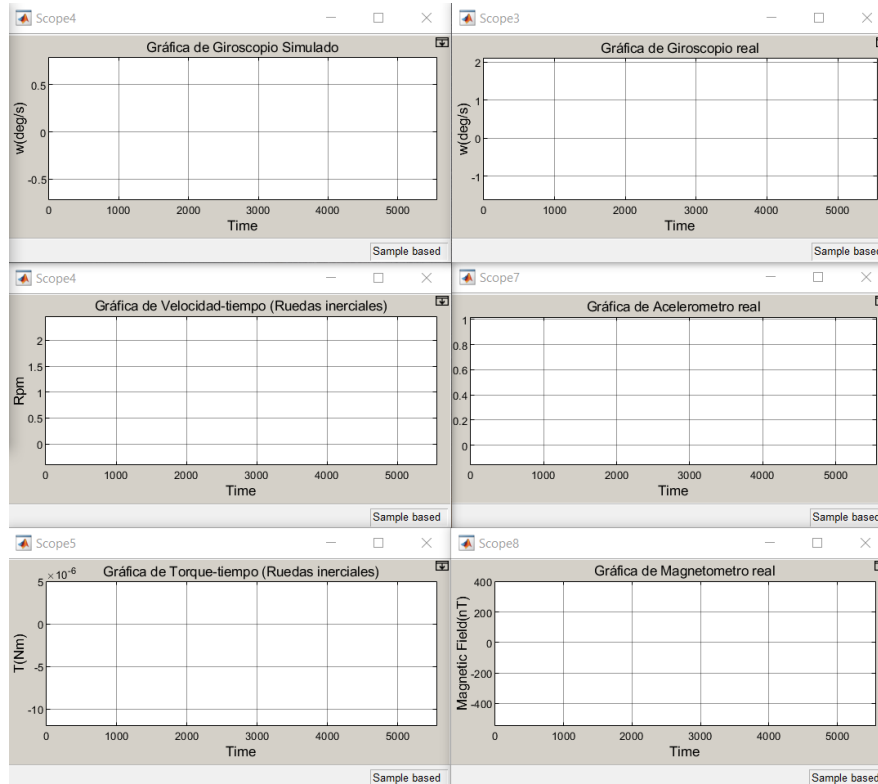


Fig. 17 Sección de Gráficas

4. Hardware In the Loop

En este reporte se emplea la plataforma inercial de seis grados de libertad mencionada con anterioridad para realizar pruebas de orientación en pequeños satélites llamados CubeSat's. A este hexápodo se le han añadido sensores de orientación externos que hacen la función de medir el movimiento de orientación asociado directamente a un Cubesat y que son operados a través de uso microcontrolador de los denominados Arduino.

4.1 Hardware Arduino

Arduino es una plataforma electrónica de código abierto para la creación de prototipos basada en software y hardware de uso sencillo. Las placas Arduino como la que se muestra en la Fig. 18 [7] pueden leer entradas analógicas o digitales (v.g. un sensor de luz, un botón o un mensaje de internet) y que, debidamente procesadas pueden producir una salida (v.g. encender un LED, activar un servomotor o para activar un motor).



Fig. 18 Placa Arduino UNO

Un usuario puede enviar un conjunto de instrucciones del microcontrolador de la placa a través del Entorno de Desarrollo Integrado (IDE) de Arduino, llamado El Software Arduino utilizando lenguaje de programación Arduino que es parecido a utilizar lenguaje C, el programa que contiene las instrucciones es llamado boceto (Ver Fig. 19), que posteriormente se compila y luego es cargado en el microcontrolador por el software Arduino.

```
File Edit Sketch Tools Help
Blink $
/*
 * Blink
 *
 * Turns an LED on for one second, then off for one second, repeatedly.
 */

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000); // wait for a second
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
  delay(1000); // wait for a second
}
1 Arduino/Genuino Uno
```

Fig. 19 Software Arduino

Todas las placas Arduino son completamente de código abierto, lo que permite a los usuarios construirlas de forma independiente y eventualmente adaptarlas a sus necesidades particulares. El software también es de código abierto y está creciendo a través de las contribuciones de los usuarios de todo el mundo.

Arduino ofrece algunas ventajas a los usuarios sobre muchos otros microcontroladores y plataformas de microcontroladores que se mencionan a continuación:

Económico: Las placas Arduino son relativamente baratas en comparación con otras plataformas. Como ejemplo el Arduino Mega2560 que será utilizará en este proyecto [8] el precio ronda los 35 euros (\$780 MXN) en el sitio web oficial.

Multiplataforma: El software de Arduino (IDE) se ejecuta en Windows, Macintosh OS X, y Linux.

Entorno de programación simple y claro: El software Arduino es fácil de usar para principiantes, pero lo suficientemente flexible como para que los usuarios avanzados también lo aprovechen. Para los maestros, está basado en el entorno de programación de procesamiento, por lo que los estudiantes que aprendan a programar en ese entorno estarán familiarizados con el funcionamiento del IDE de Arduino.

Código abierto, software y hardware extensible: el código fuente del software Arduino se publica como herramientas de código abierto. Los diseños de referencia de hardware de Arduino también están disponibles en el sitio web de Arduino. El lenguaje se puede ampliar a través de las bibliotecas de C ++, y las personas que deseen comprender los detalles técnicos pueden dar el salto de Arduino al lenguaje de programación AVR C en el que se basa. Del mismo modo, puede agregar el código AVR-C directamente en sus programas Arduino si lo desea.

Especificaciones generales

- La placa Arduino Mega2560 se basa en el microcontrolador ATmega2560 de 8 bits.
- El voltaje de funcionamiento nominal del Arduino Mega2560 es de 5V. La alimentación puede ser suministrada ya sea desde el conector de alimentación de corriente directa o desde el pin VIN de la placa, con un voltaje de entrada recomendado de entre 7V a 12V. También puede ser alimentado por el conector USB. El pin de 5V y el pin de 3.3V pueden proporcionar una fuente de alimentación regulada a otros componentes electrónicos.
- El reloj del dispositivo es proporcionado por un oscilador de cristal de 16MHz.
- El Arduino Mega2560 cuenta con 54 pines digitales (pin 0 - 53) y 16 pines analógicos (pin A0 - A15). Cada pin digital puede servir como entrada o salida, operando a 5V. Los pines analógicos se pueden usar para leer el voltaje de entrada en el pin y mapear los voltajes entre 0 y el voltaje de referencia en valores enteros entre 0 y 1023 (10 bits).
- La corriente proporcionada por cada pin E/S es de 40mA y la proporcionada por el pin de 3.3V es de 50mA.

Además, también se tiene algunos pines digitales tienen funciones especializadas, como se muestra en la Fig. 20 y que a continuación se describen.

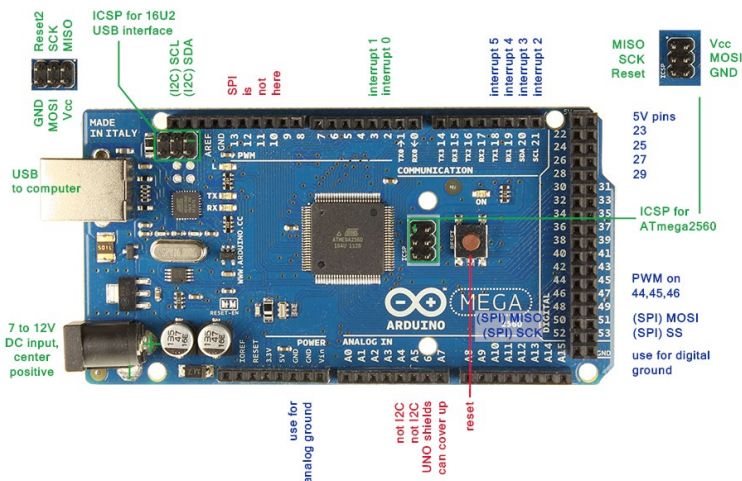


Fig. 20 Arduino Mega2560

- **Interrupciones externas**

En Arduino, una interrupción se puede activar normalmente por cuatro tipos de estado de voltaje en el pin: un nivel BAJO, un CAMBIO de nivel, el borde RISING o el borde FALLING del nivel de voltaje. En lugar de requerir continuamente el estado del pin en el programa, se detecta una interrupción automáticamente y un evento definido por el usuario puede ejecutarse inmediatamente. El Arduino Mega2560 proporciona 6 pines de interrupción externos: 2 (interrupción 0), 3 (interrupción 1), 21 (interrupción 2), 20 (interrupción 3), 19 (interrupción 4), 18 (interrupción 5). (Ver Fig. 20).

- **Modulación de ancho de pulso (PWM)**

Como pin PWM de salida, la parte del tiempo que la señal de salida pasa ON (5V) contra el tiempo que la señal pasa OFF (0V) puede controlarse. La parte del "tiempo de encendido" contra el tiempo para un ciclo completo de encendido y apagado (2 ms para las placas Arduino) se llama ciclo de trabajo. Por ejemplo, puede controlar el brillo del LED conectado al pin variando el ciclo de trabajo. El Arduino Mega2560 tiene como PWM los pines 2 - 13 y 44 - 46 que puede proporcionar salidas PWM de 8bit, que solicita un ciclo de trabajo entre 0% y 100% al escalar entre 0 y 255.

- **Comunicación Serial**

Receptor / transmisor asíncrono universal (UART): Todas las placas Arduino tienen al menos un puerto UART de serie. Se comunica con la computadora por USB o con otras interfaces mediante los pines digitales 0 (RX0) y 1 (TX0), a un nivel TTL (lógica de transistor-transistor) de 5V (3.3V para una placa de 3.3V). El Arduino Mega2560 tiene tres puertos UART adicionales: Serie1 (TX1 / RX1), Serie2 (TX2 / RX2) y Serie3 (TX3 / RX3).

Interfaz periférica en serie (SPI): El lado maestro está generando una señal de reloj (SCK) para sincronizar la transmisión de datos entre dispositivos. Una vez que un esclavo es despertado por las señales de la línea SS (selección de esclavo), los datos pueden transmitirse del maestro al esclavo a través de la línea MOSI (Master Out Slave In) o del esclavo al maestro a través de la línea MISO (Master In Slave Out). El protocolo SPI solo permite un maestro en el bus, pero admite varios esclavos. El Arduino Mega2560 proporciona el pin 51 (MOSI), el pin 50 (MISO), el pin 52 (SCK) y el pin 53 (SS) para las comunicaciones SPI. (Ver Fig. 20).

Circuito Inter-Integrado (I²C): El bus I²C puede admitir múltiples esclavos y múltiples maestros, con velocidades de datos y complejidades de hardware que caen entre SPI y UART. Cada bus I²C consta de dos líneas de señal, la señal de reloj (SCL) y la señal de datos (SDA). Todos los dispositivos en el bus comparten un reloj común generado por el maestro actual. Los datos se transmiten entre el maestro actual y el esclavo que se indica mediante el marco de dirección. El Arduino Mega2560 proporciona el pin 20 (SDA) y el pin 21 (SCL) para las comunicaciones I²C.

- **Memoria**

Los microcontroladores Atmel AVR normalmente utilizados en las placas Arduino tienen tres grupos de memoria:

Memoria Flash: aquí es donde se almacena el boceto Arduino y en el caso del Arduino Mega es de 256 kB. Es una memoria no volátil que puede recuperar datos almacenados incluso después de haber sido sintonizado y luego encendido.

Memoria estática de acceso aleatorio (SRAM): aquí es donde el boceto crea y manipula variables cuando se ejecuta. Es una memoria volátil donde los datos se pueden mantener solo cuando está encendido, el Arduino Mega contiene 8 kB.

Memoria de solo lectura programable y borrrable eléctricamente (EEPROM): es una memoria no volátil que los programadores pueden usar para almacenar información a largo plazo, el Arduino Mega contiene solo 4 kB.

4.2 Sensores de orientación

Actualmente existe una gran variedad de sensores para la determinación de la orientación de un satélite. Estos se pueden clasificar en 2 categorías:

Sensores de referencia: Otorgan una orientación mediante la medición de la dirección respecto de un objeto, una estrella, la Tierra o inclusive el sol.

- Sensores de estrellas
- Magnetómetros
- Sensores de horizonte
- Sensores solares
- GPS

Sensores de inercia: Miden continuamente los cambios en posición, sin embargo, necesitan un ajuste y una calibración de los sensores de referencia. Los giroscopios representan el único sensor conocido hasta ahora como sensor de inercia.

Para realizar una buena orientación del satélite se deben combinar ambas categorías de sensores, de referencia y de inercia, ya que es difícil que baste un solo tipo de sensor para medir adecuadamente la orientación de un satélite.

Para la implementación de la interfaz con el HIL se trabaja con un sensor comercial que integra giroscopio, acelerómetro y magnetómetro.

4.2.1 Sistema de navegación inercial

Un sistema de navegación inercial es un sistema de ayuda para la navegación que utiliza las propiedades inerciales de los sensores montados a bordo del vehículo como por ejemplo acelerómetros o giroscopios, para calcular una estimación de la posición y orientación de un

objeto en movimiento [9]. Se utilizan en muchos sistemas móviles tales como: vehículos terrestres, aeronaves, submarinos, vehículos espaciales, satélites y misiles guiados. Sin embargo, su complejidad restringe los ambientes en donde pueden emplearse.

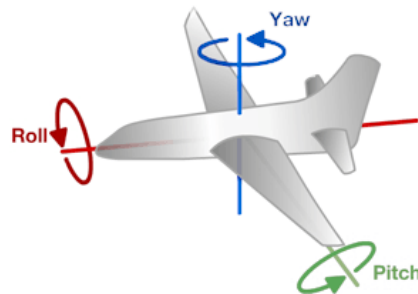


Fig. 21 Rotaciones

Los ángulos de Euler son los ángulos más utilizados para la representación de la orientación ya que son una forma intuitiva de expresar la orientación de un marco de referencia a otro marco de referencia principalmente a través de rotaciones de Roll, Pitch y n Yaw como se muestra en la Fig. 21.

En satélites se recomienda el uso de sensores de calificación para ambientes espaciales, pero estos suelen ser muy caros, espaciosos, pesados y con un consumo de energía considerablemente alta. Es por ello que se está recurriendo a sensores de tipo comercial de calificación industrial.

4.2.2 GY-85

El Sensor GY-85 como se muestra en la Fig. 22 consiste en un dispositivo con 9 grados de libertad que contiene un giroscopio MEMS de 3 ejes con resolución de 16-bits, un acelerómetro triaxial con medición de hasta ± 16 g con resolución de 13-bits y un magnetómetro de 3 ejes con medición de hasta ± 8 Gauss con una resolución de 12-bits. Admite niveles de 3.0V a 5.0V, se comunica a través del protocolo de comunicación I²C por los pines SCL y SDA. Para obtener los 9 grados de libertad se incorpora los chips ITG3205 [10] giroscopio, el ADXL345 [11] acelerómetro y el HMC5883L [12] magnetómetro cuyas características se describirán a continuación.

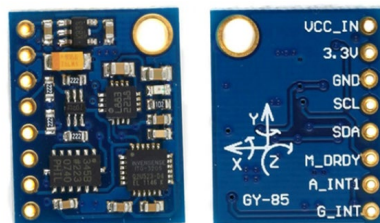


Fig. 22 Sensor GY-85

Giroscopio (ITG3205)

- Sensores de velocidad angular (giroscopios) de ejes X, Y y Z, de salida digital en un circuito integrado con un rango de escala completa de ± 2000 °/s
- Tres ADC integrados de 16 bits proporcionan muestreo simultáneo de giroscopios sin requerir multiplexor externo.

- El sesgo mejorado y la estabilidad de la temperatura de sensibilidad reduce la necesidad de calibración del usuario.
- Ruido de 0.7 °/s-rms más bajo que otros dispositivos.
- Filtro paso bajo programable digitalmente, no se necesita filtro paso alto.
- Factor de escala calibrado de fábrica.
- Interfaz I²C (400kHz).
- FRECUENCIA DE MUESTREO MAXIMA de

Acelerómetro (ADXL345)

- Resolución completa, donde la resolución aumenta con el rango g, resolución de hasta 13 bits a ± 16 g (manteniendo un factor de escala de 4 mg / LSB en todos los rangos g)
- El sistema de gestión de memoria integrada con patente pendiente con tecnología FIFO minimiza la carga del procesador host Detección de un solo toque / doble toque
- Monitoreo de actividad / inactividad
- Detección de caída libre
- Rango de voltaje de E / S: 1.7 V a VS
- SPI (3 y 4 hilos) e interfaces digitales I²C
- Modos de interrupción flexibles asignables a cualquier pin de interrupción
- Rangos de medición seleccionables mediante comando en serie
- Ancho de banda seleccionable mediante comando en serie
- 10.000 g de supervivencia al choque

Magnetómetro (HMC5883L)

- Sensores magnetorresistivos de 3 ejes, con capacidad de hasta ± 8 Gauss y un ADC de 12 bits acoplado con sensores AMR de bajo ruido logrando una resolución de campo de 2 mili-gauss.
- Velocidad de salida máxima rápida de 160 Hz.
- Habilita la prueba de funcionalidad de bajo costo después del ensamblaje en producción
- Operaciones de bajo voltaje (2.16 a 3.6V) y bajo consumo de energía (100 μ A)
- Interfaz digital I²C: Interfaz de datos en serie de dos hilos popular para electrónica de consumo.
- Amplio rango de campo magnético: Los sensores se pueden usar en entornos de campo magnético fuerte con una precisión de rumbo de brújula de 1 ° a 2 °
- Soporte de software y algoritmo disponible: Dirección de brújula, hierro duro, hierro suave y calibración automática con varias bibliotecas disponibles.

4.2.3 Estimación de la orientación

Los giroscopios miden la velocidad angular, los cuales se integran para obtener la orientación del satélite, pero suelen ser susceptibles a deriva o acumulación de error a medida que transcurre el tiempo. Por otra parte, los acelerómetros sensan el cambio en la orientación con respecto al vector de la fuerza gravitacional, pero tienden a distorsionar la medida cuando actúan fuerzas externas.

La fusión de las mediciones del giroscopio y del acelerómetro busca mejorar la medición de la orientación. Sin embargo, el acelerómetro puede ser utilizado solo para medir la rotación alrededor de los ejes x(Roll) y y(Pitch) y no alrededor del eje z(Yaw), debido a que el vector gravitacional es paralelo al eje z. Por lo tanto, se utiliza el magnetómetro para medir el ángulo

de rotación alrededor del eje z, dado que el campo magnético de la tierra posee componentes tanto en x como en z [13].

Como resultado de la integración del giroscopio, los errores de medición asociados a las mediciones del giroscopio se propagarán al ángulo estimado. Para corregir este error es necesario implementar un algoritmo de fusión y que, por ser el óptimo, es común utilizar el filtro Kalman. Este filtro en el espacio de estados se ilustra en el diagrama de la Fig. 23.

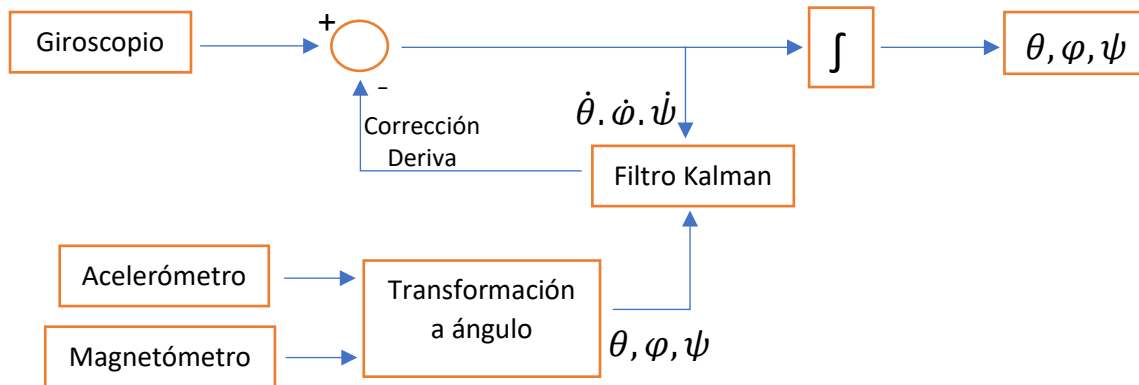


Fig. 23 Algoritmo de fusión de sensores

5. Implementación general del HIL Simulator

El HIL simulator está dividido en ocho diferentes módulos los cuales se muestran integrados en el esquema general de la Fig. 24. EL módulo central de manera natural es precisamente el que define paramétricamente el Cubesat y cuya dinámica será controlada por el módulo correspondiente. Los módulos de sensores y actuadores ayudan al módulo de control que producirá la dinámica deseada. Por el momento, y con fines de prueba se ha implementado un sistema de control PID aunque precisamente la modularidad del simulador permite la sustitución por un sistema de control más avanzado para adecuarse a las necesidades de Ingeniería. El modelo para el sistema de control se completa por medio del módulo de perturbaciones que afectan al ambiente del Cubesat. Del módulo de sensores, tanto virtuales como reales, también se obtienen los datos tanto para el despliegue de las gráficas, como para el módulo de cinemática. La información cinemática permite la visualización por medio del módulo de animación. El HIL será finalmente constituido por el módulo del Hexápodo y por ende, aquí solo se explicará el módulo del Hexápodo y el de los Sensores como la parte base de estudio para este reporte.

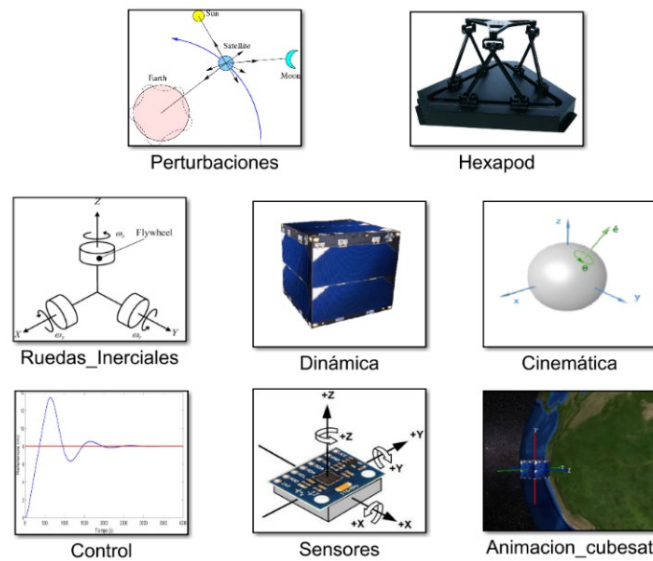


Fig. 24 Modelo Simulink

El HIL simulator funciona como un todo y es esquematizado en la Fig. 25 y empieza por el primer bloque del esquema que permite la introducción (ventana de comandos descrita en sección 3.3) de la orientación deseada; luego, se realiza el cálculo de error utilizando el álgebra de cuaterniones entre la orientación deseada y la estimación actual de la orientación (proveniente ya sea por los sensores virtuales o de los sensores reales), posteriormente se realiza el control de la orientación por medio del control PID que actúa a través del torque provisto por ruedas de inercia. A estos torques se le suman las fuerzas de perturbaciones simuladas por medio de modelos matemáticos, para que así a través de la dinámica del satélite se obtenga la orientación actual del satélite. Finalmente, la orientación actual se puede visualizar ya sea, tanto a través de la ventana de animación (descrita en la sección 3.3) como de manera real a través del Hexápodo.

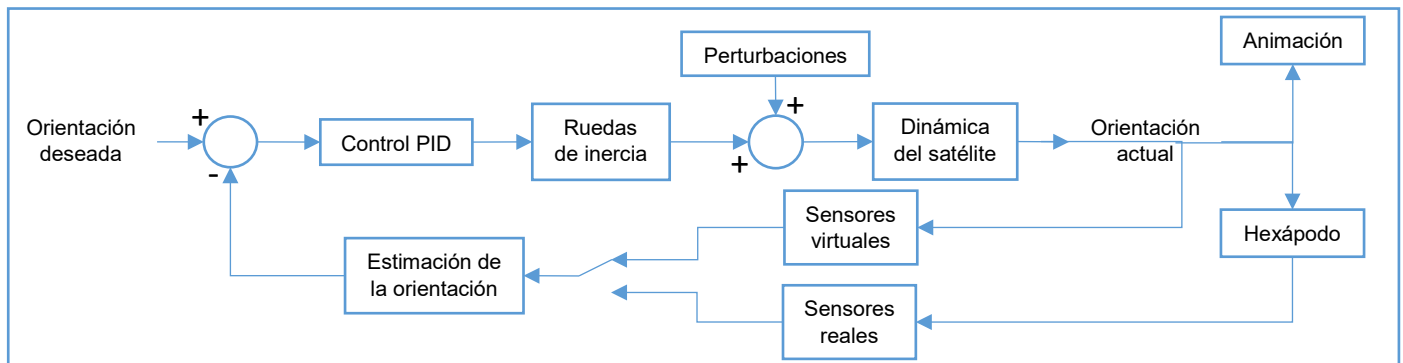


Fig. 25 Esquema general

5.1 Módulo Hexápodo

En la Fig. 26 se muestra el diagrama de bloques del funcionamiento del Hexápodo dentro del simulador. La implementación emplea directamente el mismo modelo de Simulink Hexapod_World_Control.mdl aunque solo con la variación de las tres rotaciones Roll, Pitch y Yaw, ya que por el momento para pruebas de rotación no se necesitan los movimientos lineales del Hexápodo por lo que se opta por mantenerlos en una posición de reposo.

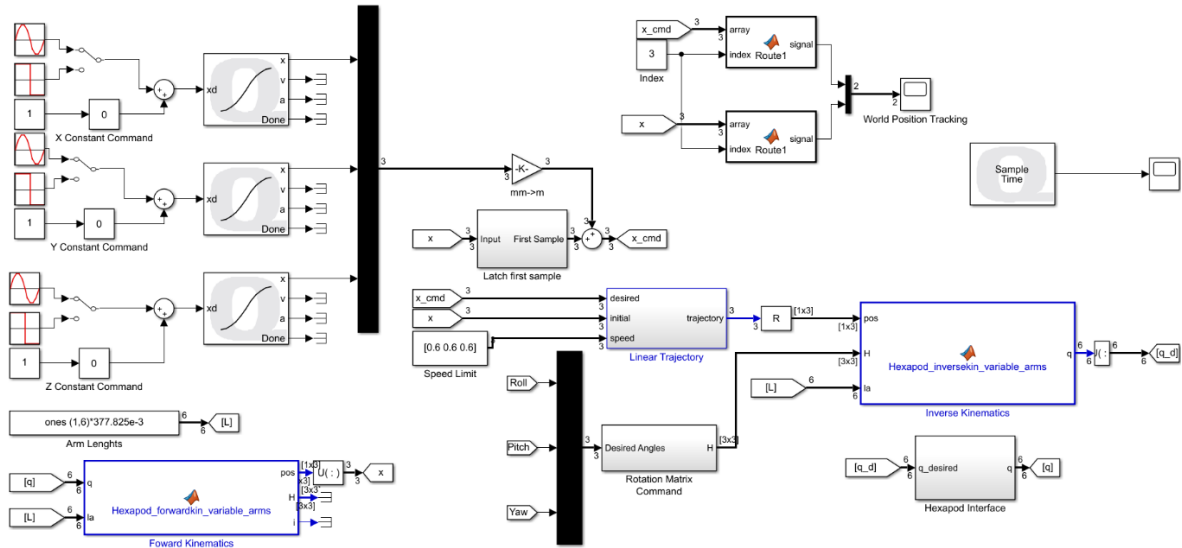


Fig. 26 Bloque del Hexápodo

5.2 Giroscopio Virtual

El siguiente bloque (Ver Fig. 27) describe el giroscopio virtual que se ocupa para el cálculo de la velocidad angular que se genera al rotar un ángulo en el Hexápodo.

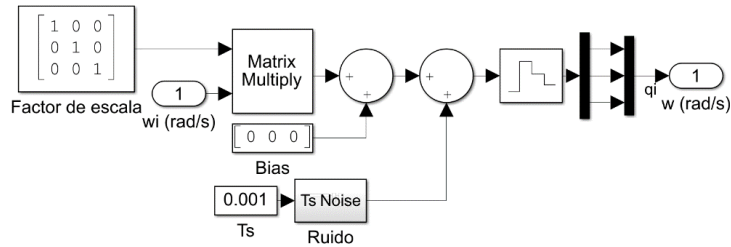


Fig. 27 Bloque de sensor virtual

El modelo del giroscopio esta dado por la siguiente ecuación:

$$\omega = (I + k_g)\omega_g + \beta_g + \eta_g$$

Donde ω es la salida del giroscopio, ω_g es la velocidad angular real del giroscopio, I es la matriz identidad, k_g es la matriz de factores de escala y desalineaciones, β_g es la deriva del giroscopio, η_g es ruido blanco gaussiano [14]. La ecuación anterior está representada en bloques en la Fig. 27, el valor de la deriva del giroscopio se considera 0 para los tres ejes, así como los factores de escala solo se considera la escala real de la medición, el ruido esta por el orden de $1e-5^\circ/\sqrt{\text{hr}}$.

5.3 IMU Real

El bloque (Ver Fig. 28) realiza la lectura de los datos de los sensores del IMU del Arduino a través de la función Stream Call leyendo directamente desde el puerto COM 3 conectado a

Es importante mencionar que cuando se ejecuta el programa los sensores que estarán activados serán los sensores virtuales, ya que los sensores reales generan lecturas erróneas al iniciar la simulación, por lo que es recomendable que siempre se inicie los sensores virtuales para no generar un problema o dañar el Hexápodo, ya que puede moverse al iniciar la simulación a una posición que esta fuera del límite de movimiento del Hexápodo. Posteriormente, el usuario puede cambiar a los sensores deseados precisamente a través de la interfaz HIL y cuyo bloque funcional se muestra en la Fig.29.



6.1 Verificación de funcionamiento del Hexápodo

26

la amplitud y la frecuencia de una onda sinusoidal o cuadrada que se pueden ajustar para cada uno de los ejes del Hexápodo ya sea X, Y, Z o los ángulos Pitch, Roll y Yaw.

Antes de ejecutar este modelo de Simulink, el Hexápodo debe estar en la posición INICIO. Si las articulaciones del Hexápodo no están en la posición central, entonces realice el procedimiento de referencia en la Sección 2.3.

Se realizaron estos pasos para que el modelo funcionara de manera correcta:

1. Se aseguró de que el botón rojo del interruptor de parada de emergencia estuviera en la posición “arriba” para habilitar el amplificador.
2. Antes de iniciar, primero se calibró el hexápodo.
3. Posteriormente se colocó a posición de INICIO como se mencionó en la Sección 2.3
4. Se ejecuta “setup_hexapod.m” en MATLAB (Fig. 7).
5. Se abre el modelo Simulink llamado Hexapod_World_Control.mdl, (Fig. 30).
6. Se hace clic en Build model para generar el controlador.
7. Se ejecuta el modelo de Simulink dando clic en Connect To Target y posteriormente en Start.
8. Se cambiaron los valores para las rotaciones Roll, Pitch y Yaw y se observó como el Hexápodo empieza a moverse (Fig. 31)

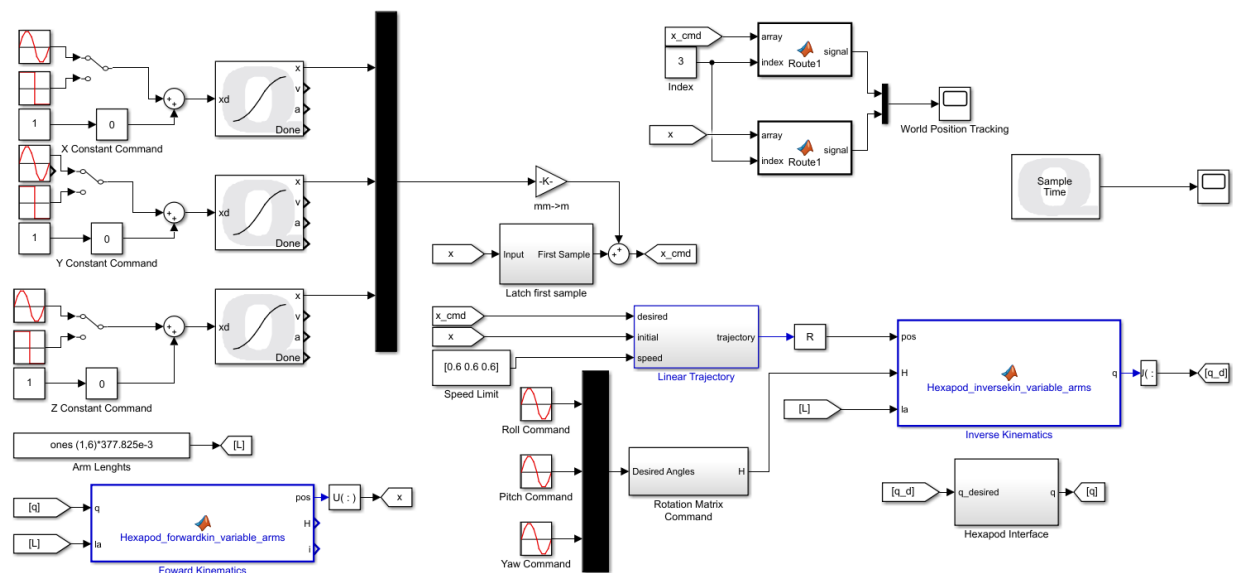


Fig. 30 Hexapod_World_Control.mdl

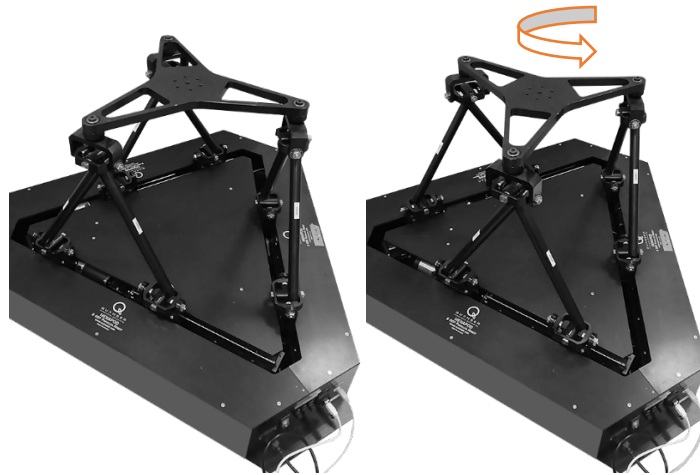


Fig. 31 Movimiento Yaw del Hexápodo

6.2 Instalación y operación del IMU

Se instaló y opero el sensor inercial GY-85 descrito en secciones anteriores en la plataforma como se muestra en la Fig. 32. el IMU se colocó de tal forma que el sensor quedara lo más centrado posible en la plataforma, se realizaron pruebas de orientación al Hexápodo implementando.

Las pruebas que se realizaron permiten conocer los datos obtenidos por los sensores teniendo en cuenta que el Hexápodo estará en posición de reposo y con diferentes orientaciones.

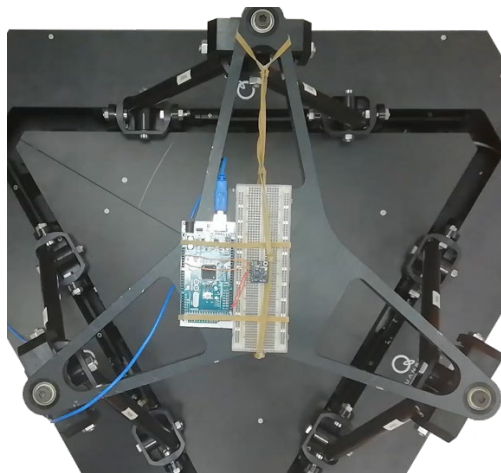


Fig. 32 Hexápodo con sensor inercial

Sensores reales

Para la primera prueba la posición que se desea es que el Hexápodo mantenga los tres ángulos de rotación en 0° por lo que permanecerá sin moverse, pero por causa de vibraciones existentes dentro de los mecanismos del Hexápodo, este tendrá pequeñas vibraciones alrededor de $\pm 0.05^\circ$ que son imperceptibles para un usuario pero si detectables para el sensor real. Estas vibraciones del sistema real pueden considerarse como una perturbación ambiental y debe ser sumada al ruido existente de los sensores.

Los datos obtenidos por estos sensores reales se grafican en la Fig. 33. Para las rotaciones Roll y Pitch puede apreciarse que las oscilaciones totales están dentro de un margen de $\pm 0.2^\circ$ ocasionadas por el ruido presente en los sensores y las vibraciones ambientales. Para la rotación Yaw, que es afectada por el campo magnético terrestre sucede algo diferente ya que éste varía con el tiempo por lo que sus valores oscilan aproximadamente en $\pm 0.4^\circ$ quedando así establecido que la variación de esta rotación será considerablemente más grande en comparación con las otras dos rotaciones.

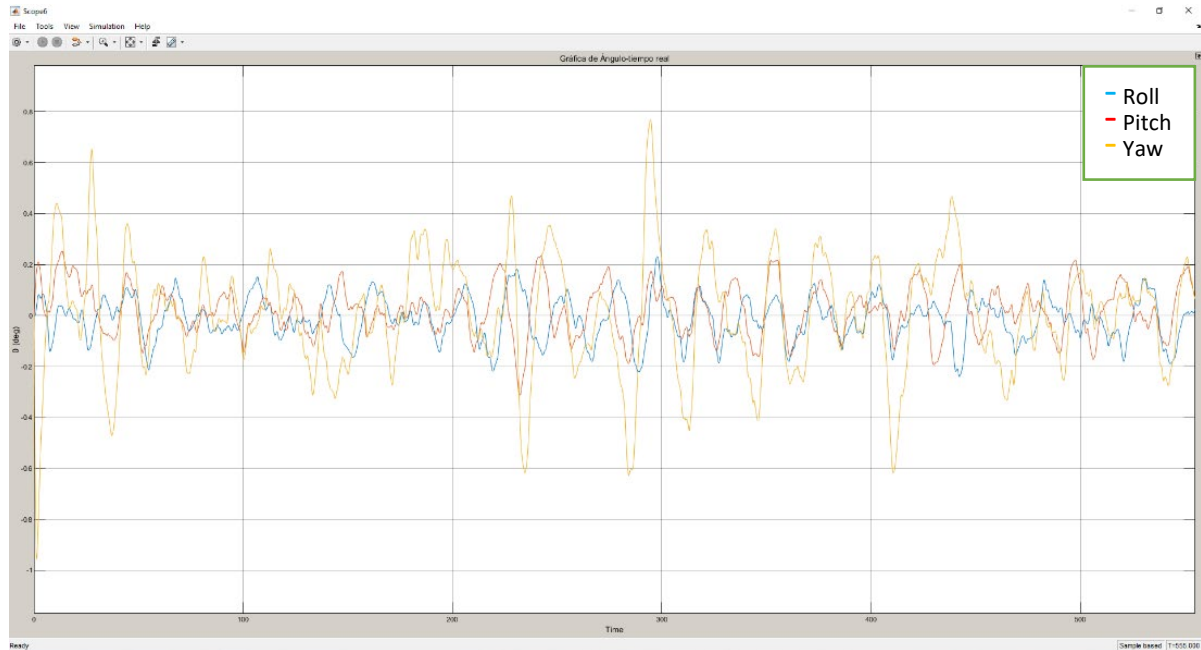


Fig. 33 Gráfica de la orientación del satélite en posición estable con sensores reales

Para la segunda prueba, se desean diferentes rotaciones alrededor de los 3 ejes del Hexápodo. Dadas las limitaciones del Hexápodo descritas en la sección 2.1 estas pruebas se realizaron solo a 5° en cada eje.

Los datos obtenidos por cada rotación realizada se pueden observar en la gráfica de la Fig. 34. El ruido existente en los sensores y la vibración presente en el Hexápodo se puede apreciar por las oscilaciones de $\pm 0.2^\circ$ alrededor de los 5° tanto para Roll y Pitch mientras que para Yaw se aprecia la variación más grande por el motivo antes mencionado del campo magnético como era de esperarse.

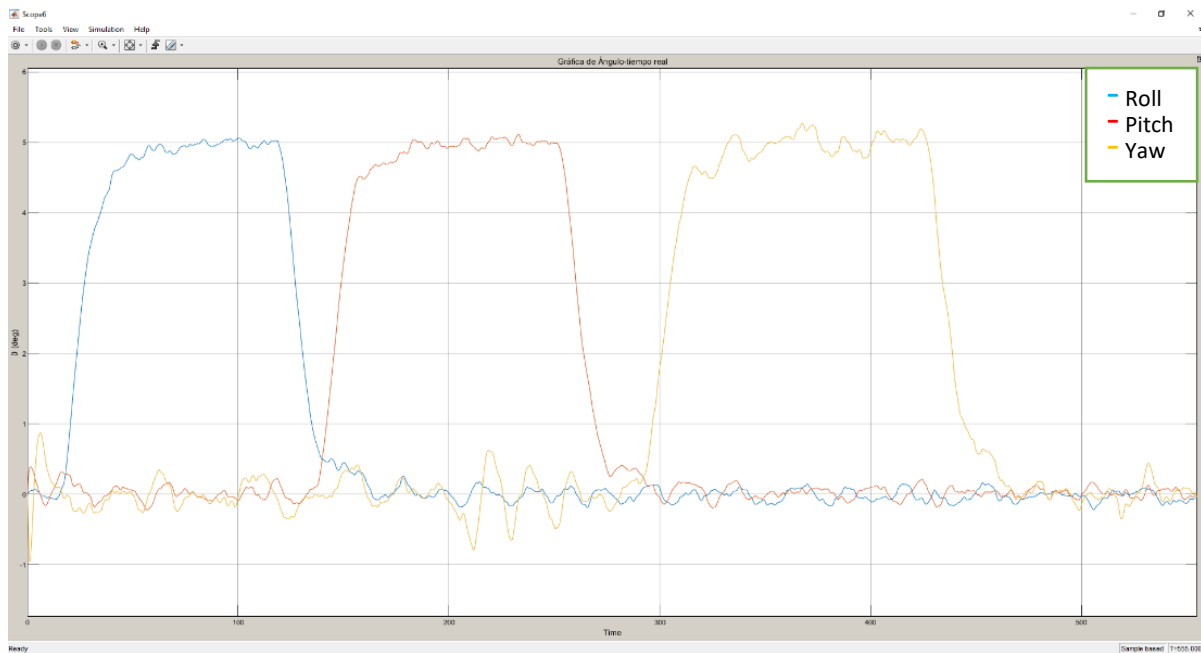


Fig.34 Gráfica de la orientación del satélite con diferentes orientaciones con sensores reales

Sensores virtuales

Se realizarán las mismas pruebas, pero ahora con el sensor virtual, en este caso solo se hará uso del giroscopio, este modelo de giroscopio proporciona medidas al que solo se le añade un ruido para representar el ruido propio de un sensor real pero no se considera un sesgo o bias presente en el giroscopio real. Así pues, en comparación con el sensor real, el ruido simulado en el giroscopio virtual es 10 veces menor que el ruido en un sensor real.

La órbita que se simuló para estas pruebas fue una órbita circular a una altura de aproximadamente 400 km con una velocidad angular orbital inicial de 0.001131 rad/s (0.064822 °/s que equivale a una velocidad tangencial 7.8 Km/s). El sistema de control de la rotación del CubeSat está activado de tal forma que el satélite esté orientado hacia el nadir, esto es, se mantiene una orientación deseada de 0°. Además, por el sistema de referencia utilizado, el satélite tiende a rotar sobre el eje correspondiente al Pitch.

Esta rotación de Pitch se puede observar en la gráfica de la Fig. 35 desde el segundo 0 al 20 de la simulación donde se aprecia la tendencia a incrementarse en comparación con el Roll y Yaw. Debe notarse también que esta rotación transitoria del Pitch no se puede observar directamente en el Hexápodo ya que el movimiento inicial no se aprecia por el hecho de que es muy pequeño, pero los sensores tanto el real como el virtual si detectan la rotación con la que inicia el simulador.

Todos los datos obtenidos por el giróscopio virtual se pueden observar en la gráfica de la Fig. 35. Como ya se mencionó por causa de vibraciones existentes dentro de los mecanismos del Hexápodo los sensores reales de rotación si las detectarán en tanto que el sensor virtual, al no considerarlas no se reflejarán en sus correspondientes mediciones.

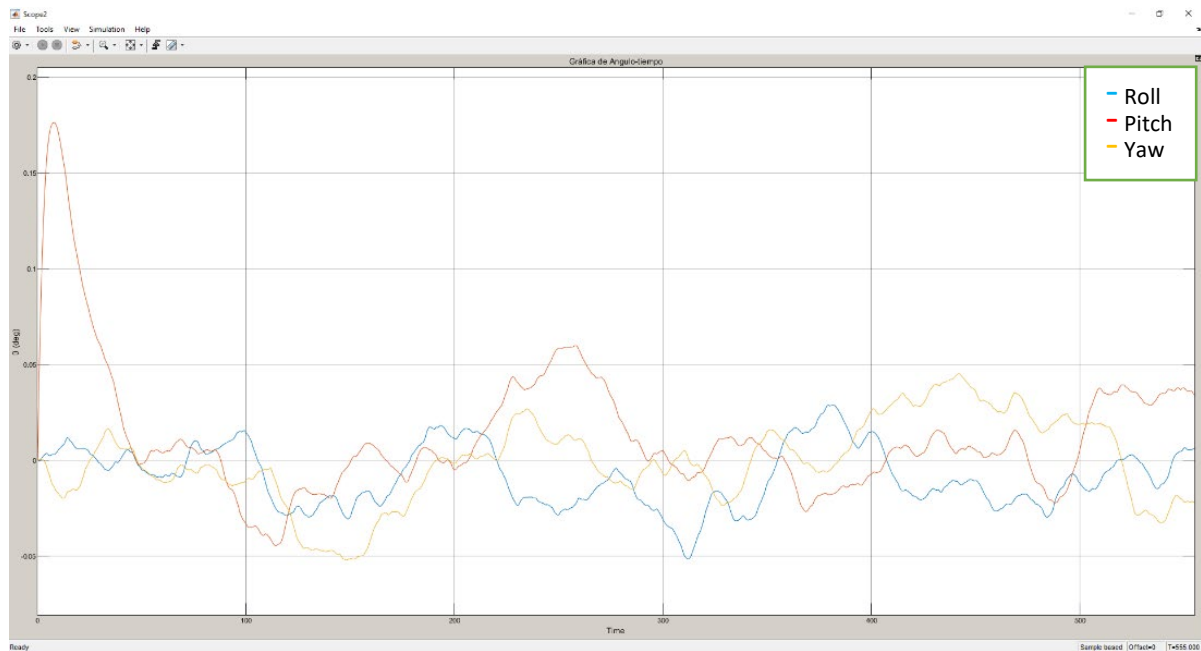


Fig. 35 Gráfica de la orientación del satélite en posición estable con sensores virtuales

Para la segunda prueba que se realiza con los sensores virtuales, al igual que los sensores reales se desean diferentes rotaciones alrededor de los 3 ejes del Hexápodo, por lo que también se realizaron pruebas de solo 5° por cada eje.

Los datos obtenidos por cada rotación realizada se pueden observar en la gráfica de la Fig. 36. Acorde con lo señalado anteriormente respecto a que el sensor virtual no considera el ruido ambiental del Hexápodo, la variación existente en cada eje es de 0.04° aproximadamente cuando se llega al objetivo de 5° , que no es tan grande en comparación con los sensores reales.

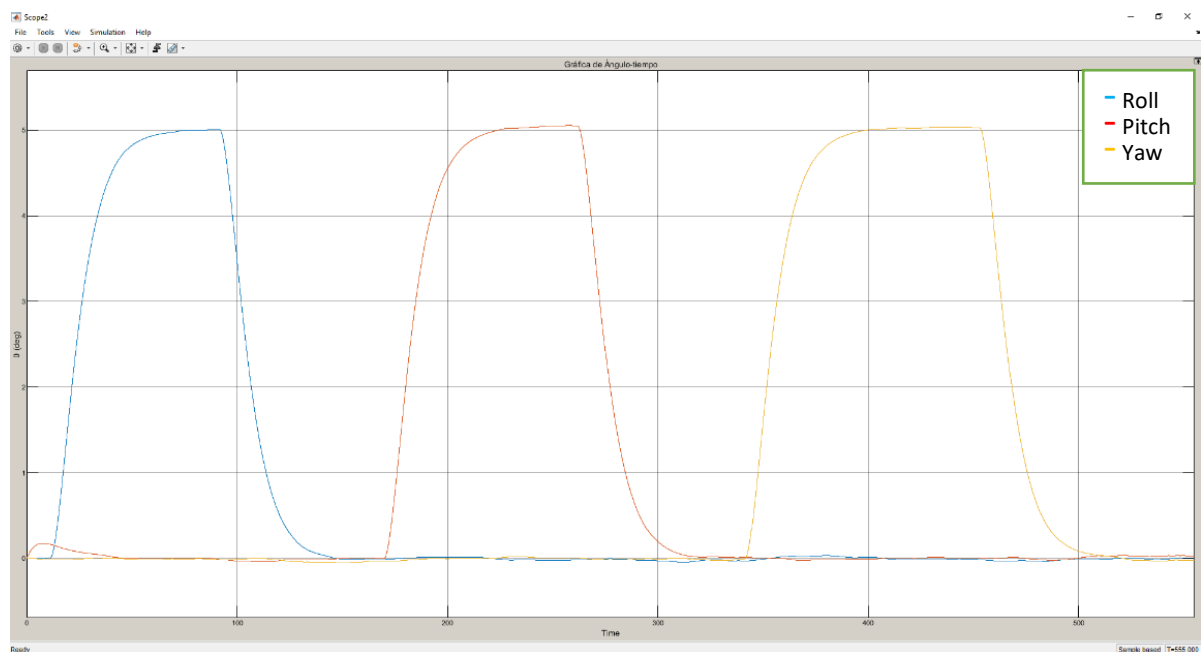


Fig.36 Gráfica de la orientación del satélite con diferentes orientaciones con sensores virtuales

Comparación de sensores

Una vez alcanzados los objetivos anteriores y obtenido los resultados correspondientes, se analizará y se compararan los resultados obtenidos por los sensores reales contra los obtenidos por los sensores simulados del Simulador.

Los resultados obtenidos por los sensores reales y virtuales en la rotación Roll se puede observar en la gráfica de la Fig.37 donde se aprecia que el sensor virtual presenta una variación de $\pm 0.02^\circ$ en 0° mientras que para el sensor real presenta una variación de aproximadamente ± 0.1 en 0° . Como era de esperarse, las variaciones en los sensores reales son mayores ya que éstos detectan las vibraciones provocadas por los mecanismos del Hexápodo así como las desviaciones existentes en el magnetómetro al momento de realizar las mediciones.

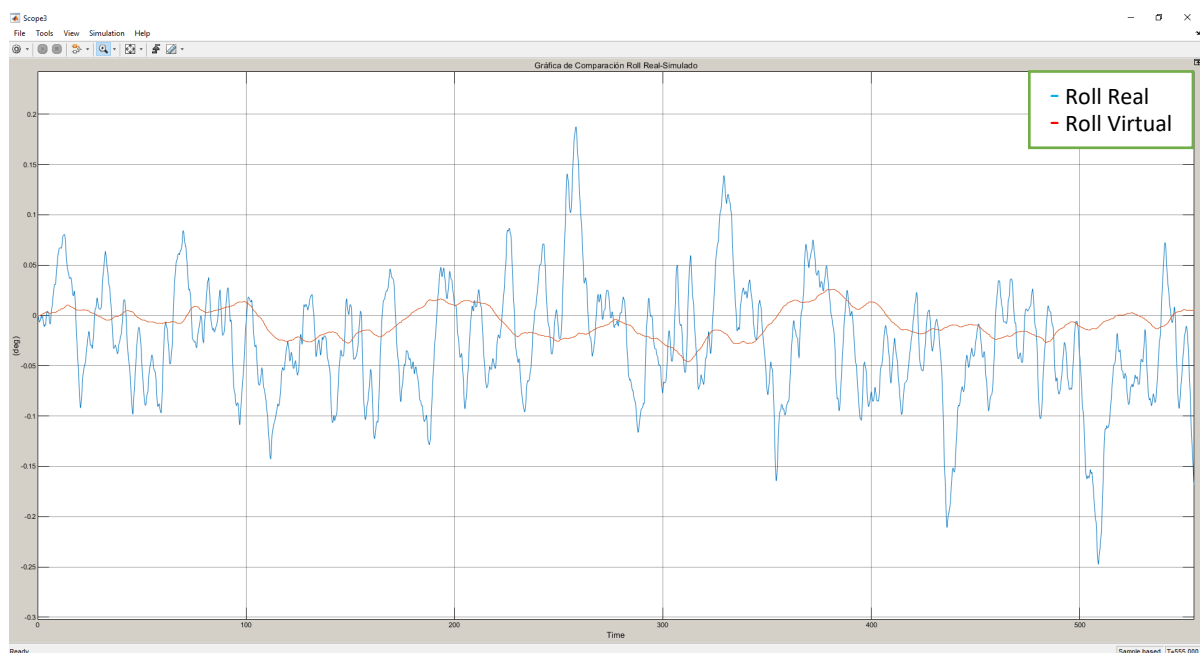


Fig. 37 Gráfica de la orientación Roll en posición estable con sensores virtuales y reales

Para el sensor virtual se tiene un error cuadrático medio de 0.00018509 y para el sensor real se tiene un error cuadrático medio de 0.002712. Como el sensor real mide las oscilaciones presentes por el Hexápodo, su error no es muy grande en comparación con el error que existe en el sensor virtual, por lo que se puede ocupar para realizar mediciones de la orientación dependiendo del objetivo de la misión para el cual la orientación es importante.

En la gráfica de la Fig. 38 se puede observar que en la rotación Pitch del segundo 0 al 20 de la simulación el sensor virtual tiende a incrementarse, esto debido a que la velocidad orbital solo está presente en Pitch y de igual manera, los sensores reales en el Hexápodo detectan la rotación provocada por la velocidad orbital.

En la Fig. 38 se aprecia que el sensor virtual presenta una variación de $\pm 0.04^\circ$ en 0° mientras que para el sensor real presenta una variación de aproximadamente ± 0.1 en 0° .

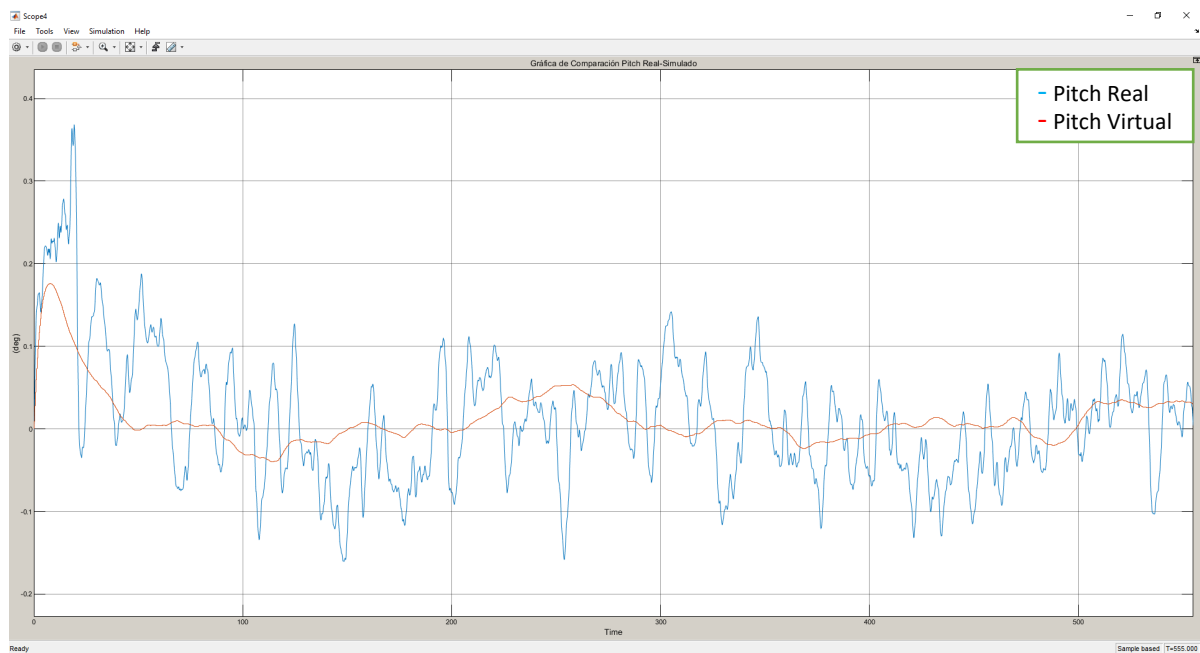


Fig. 38 Gráfica de la orientación Pitch en posición estable con sensores virtuales y reales

Para el sensor virtual se tiene un error cuadrático medio de 0.0022215 y para el sensor real se tiene un error cuadrático medio de 0.009805.

La señal Yaw real mostrada en la gráfica de la Fig.39 está compuesta por el giroscopio y el magnetómetro. Como se mencionó anteriormente existen variaciones originadas por el campo magnético de la Tierra, así como otras perturbaciones magnéticas del ambiente presentes al obtener mediciones con el magnetómetro.

Los resultados obtenidos de esta gráfica para los sensores reales y virtuales indican que el sensor virtual presenta una variación de $\pm 0.02^\circ$ en 0° mientras que para el sensor real se tiene una variación de aproximadamente ± 0.4 en 0° .

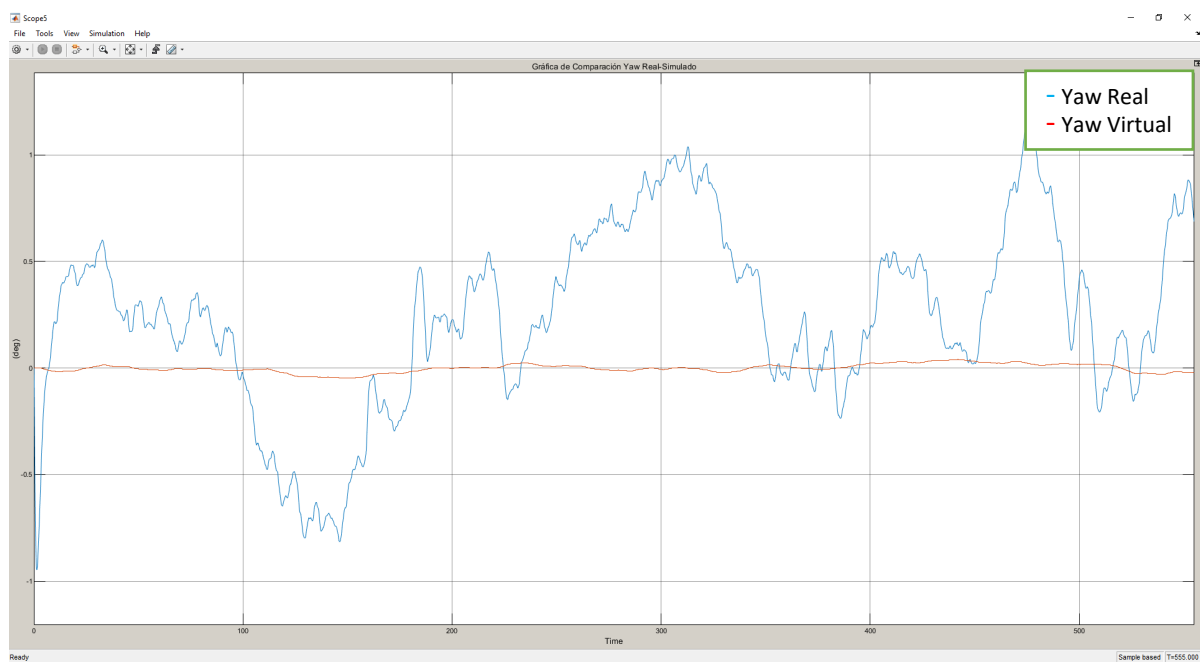


Fig. 39 Gráfica de la orientación Yaw en posición estable con sensores virtuales y reales

Para el sensor virtual se tiene un error cuadrático medio de 0.0004598 y para el sensor real se tiene un error cuadrático medio de 0.19827. Esta gran diferencia entre los 2 tipos de

sensores es debida a que el sensor virtual solo está haciendo uso del giroscopio, en cambio para el sensor real se están combinando un giroscopio y un magnetómetro. Y como el magnetómetro mide las variaciones del campo magnético en el entorno, entonces, para esta prueba la señal medida por los sensores reales no es una buena estimación para obtener la rotación Yaw.

Ahora que se tienen los resultados de la comparación entre la instrumentación virtual y la real, se podrán comparar las mediciones efectuando rotaciones en sus respectivos ejes utilizando en el análisis de las gráficas correspondiente el error cuadrático medio.

Para la primera rotación en roll en la gráfica de la Fig. 40 se observa que del segundo 150 al 555 el error se sigue manteniendo como ya se había mencionado con anterioridad, pero cuando se efectúa la rotación se aprecia que el sensor real no esta sobre los 5° al cual tenía que llegar, sino que se queda en un valor de 5.23° aproximadamente y donde tiene una variación de ± 0.1 .

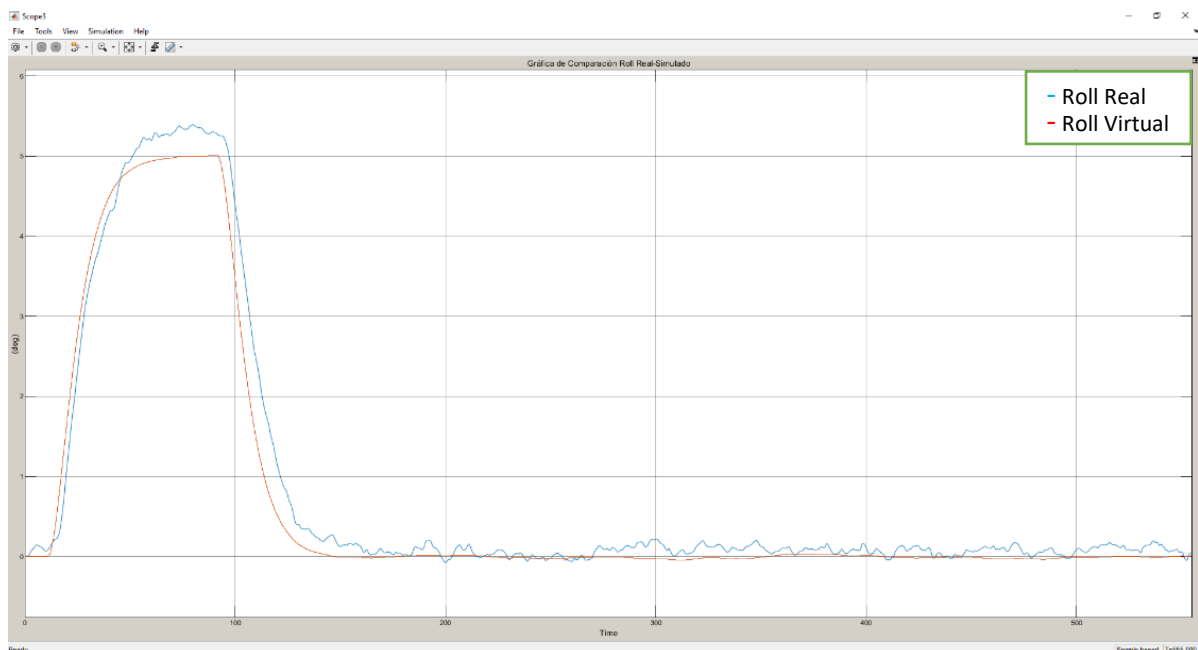


Fig. 40 Gráfica con rotación en Roll con sensores virtuales y reales

En el sensor virtual se tiene un error cuadrático medio de 0.008218 y para el sensor real se tiene un error cuadrático medio de 0.075057.

A pesar de que el sensor real sobrepasa por 0.23° los 5° que se asignaron, aún es posible mantener un control adecuado con los sensores reales, dependiendo mucho de la aplicación que se pida, el seguimiento de un objeto no es adecuada si se desea mayor precisión.

En comparación con la rotación anterior en la gráfica de la Fig. 41 se puede observar que en la rotación Pitch al momento de realizar la rotación en 5° el resultado que se obtiene es una mejor estimación que para la rotación Roll, la orientación estimada por los sensores reales es de 5.12° con una variación de ± 0.1

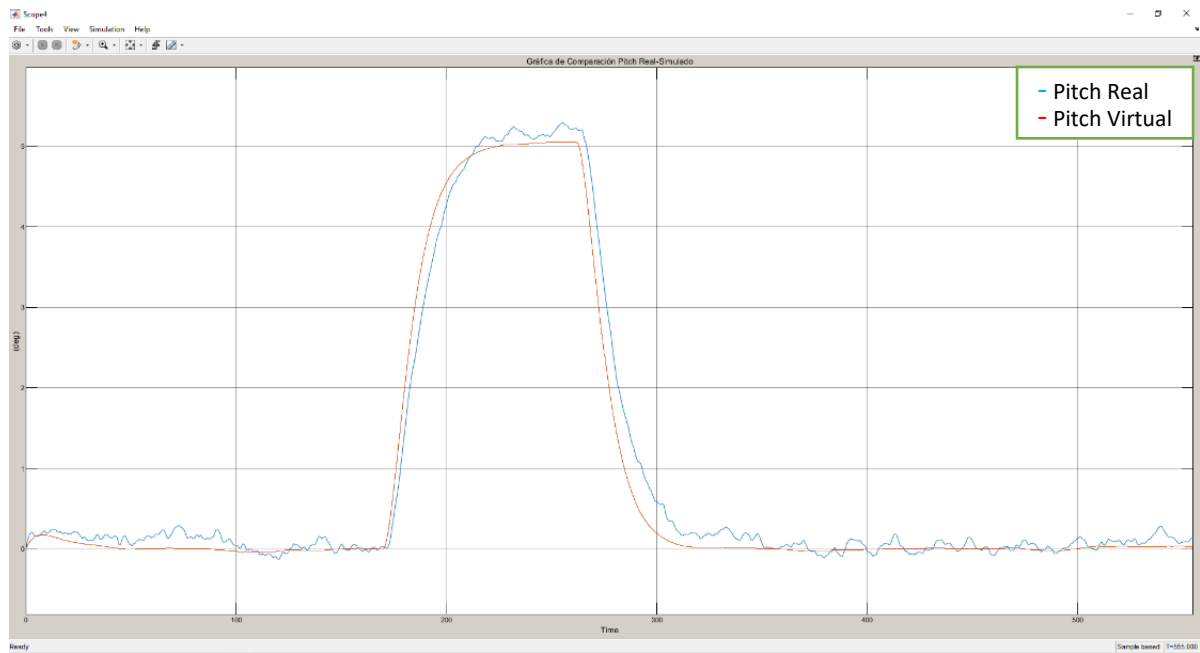


Fig. 41 Gráfica con rotación en Pitch con sensores virtuales y reales

Para el sensor virtual se tiene un error cuadrático medio de 0.005946 y para el sensor real se tiene un error cuadrático medio de 0.033599. Por lo que aún se puede considerar que el sensor real realiza una buena estimación de la orientación.

Los resultados obtenidos por los sensores reales y virtuales en la rotación yaw se puede observar en la gráfica de la Fig.42 donde se aprecia que el sensor virtual presenta una variación de $\pm 0.02^\circ$ en 0° mientras que para el sensor real presenta una variación de aproximadamente ± 1.3 en 0° , cuando se realiza la rotación de 5° el sensor virtual si llega a los 5° pero el sensor real intenta llegar, sin embargo, no logra alcanzar el objetivo pero se mantiene aproximadamente en 4.74° .

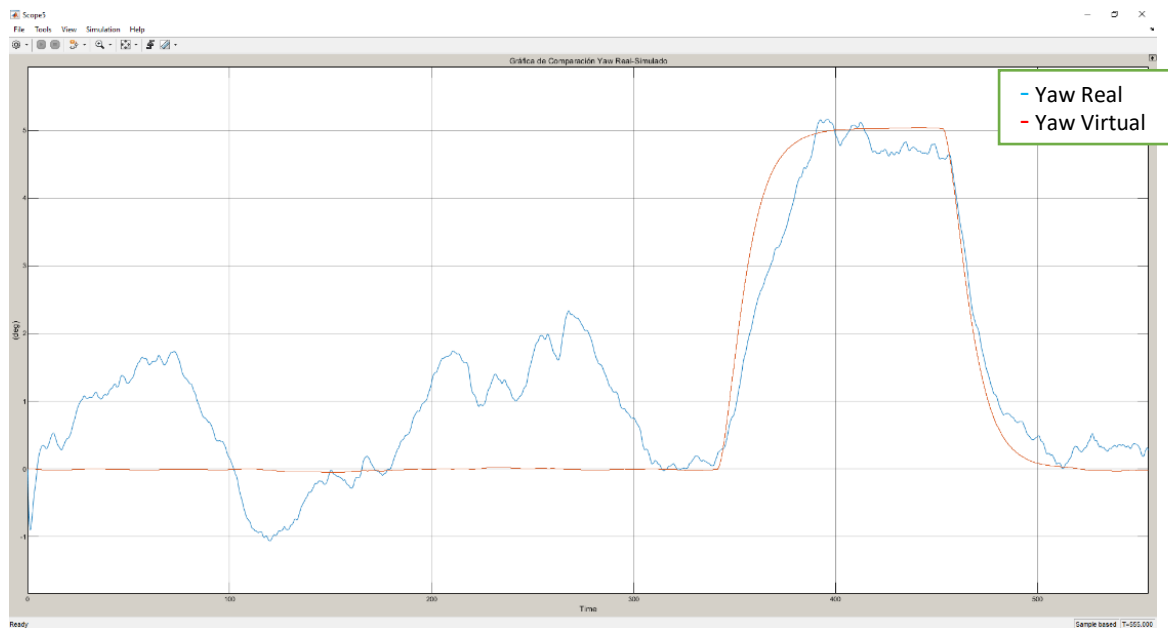


Fig. 42 Gráfica con rotación en Yaw con sensores virtuales y reales

Para el sensor virtual se tiene un error cuadrático medio de 0.004686 y para el sensor real se tiene un error cuadrático medio de 0.128662. Por lo que a pesar de que se realiza la orientación en 5° e intenta llegar la señal medida por los sensores reales, la señal medida por la fusión del giroscopio y el magnetómetro no realiza una adecuada estimación para obtener la rotación yaw.

7. Conclusiones y Recomendaciones

La implementación de un HIL ayuda a una persona a realizar una navegación básica con la GUI de un simulador después de recibir instrucciones básicas de su funcionamiento, por lo que los usuarios pueden simplemente hacer clic en un elemento con el que quieren interactuar.

El poder visualizar el satélite como se mostró en las Figs. 14 y 16 permite al usuario la interpretación física de los resultados que se muestran en las gráficas, la animación que se presenta remplace visualmente al Hexápodo al momento de realizar pruebas en la interfaz de usuario.

Los sensores reales no solo pueden detectar la vibración del Hexápodo, sino que también puede detectar vibraciones de baja frecuencia presentes en el medio ambiente, en cambio los sensores virtuales carecen de ello, sin embargo, el modelo virtual se podría mejorar, bien incorporando perturbaciones ambientales como la variación magnética en el tiempo, así como también realizar el análisis de las vibraciones ocasionadas por el Hexápodo para implementarlo en trabajos futuros.

La importancia del trabajo radica en la implementación de sensores virtuales como remplazo de los sensores reales, a pesar de que los giroscopios virtuales cuentan con ruido en las mediciones, pero no con deriva e ignorando las perturbaciones que se originan en el ambiente, así como las vibraciones causadas por el Hexápodo, se puede realizar una estimación suficientemente adecuada como para orientar al Hexápodo, por lo que se pueden reemplazar por los sensores reales para este trabajo.

Se realizó la fusión de varios sensores reales para poder realizar la estimación de la orientación, para las rotaciones roll y pitch se utilizaron giroscopios y acelerómetros lo que originó la eliminación de la deriva ocasionada por los giroscopios, a pesar del ruido existente en estos sensores la estimación de la orientación es relativamente aceptable para realizar diferentes orientaciones en el Hexápodo que no exige una orientación precisa para este trabajo, pero no es suficientemente adecuada para una aplicación que exija una orientación con mayor precisión como el seguimiento de un objeto en el espacio. Para la rotación yaw se presenta un caso especial ya que para realizar la estimación de la orientación se fusionó un giroscopio y el magnetómetro, como el campo magnético varía en el tiempo no es posible estimar la orientación con el magnetómetro con cierta precisión, a pesar de que al momento de fusionarse con el giroscopio se elimina la deriva ocasionada por este último, no se eliminan los efectos de variación del campo magnético en el tiempo, lo que origina que este tenga grandes variaciones en la orientación como se mostró en las Figuras 39 y 42, por lo que no es recomendable fusionar estos dos sensores para estimar la orientación yaw, de ser posible es mejor realizar la fusión con otro tipo de sensores como el sensor de horizonte.

El hecho de que la mayoría del trabajo está desarrollado en Simulink permite el uso de herramientas y librerías que facilitan la interconexión existente entre el Hexápodo y el simulador, el cual permite continuar con el desarrollo de pruebas e implementación y validación con otro tipo de Hardware In the Loop.

8. Referencias

- [1] C. Kleijin. Introduction to Hardware-in-the-Loop Simulation, ControlLab Products B.V
- [2] S. López Escobedo, Modelado de la orientación en órbita de CubeSats (Tesis de maestría), Instituto Nacional de Astrofísica, Óptica y Electrónica, Cholula, Puebla, octubre, 2019.
- [3] Campa, R., Bernal, J., & Soto, I. (2016). Kinematic modeling and control of the Hexapod parallel robot. *2016 American Control Conference (ACC)*, 1203-1208.
- [4] <https://www.quanser.com/products/hexapod/>
- [5] Quanser Inc. QUARC Installation Guide, 2016.
- [6] Quanser Inc. Quanser 6 DOF Hexapod User Manual, 2016.
- [7] <https://www.arduino.cc/en/Main/Software>
- [8] D. Morante G., M. Soler A., X. Chen, On-Board System, Aerospace Engineering, Universidad Carlos III de Madrid, 2017
- [9] Oliver J. Woodman, An introduction to inertial navigation (Technical Report), University of Cambridge, August 2007
- [10] http://dl.btc.pl/kamami_wa/itg3205.pdf
- [11] <https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL345.pdf>
- [12] https://cdn-shop.adafruit.com/datasheets/HMC5883L_3-Axis_Digital_Compass_IC.pdf
- [13] Kaiqiang Feng, Jie Li. A New Quaternion-Based Kalman Filter for Real-Time Attitude Estimation Using the Two-Step Geometrically-Intuitive Correction Algorithm, North University of China, September 2017.
- [14] L. W. J. L. W. Y. Liang Xue, Chengyu Jiang. Noise reduction of mems gyroscope based on direct modeling for an angular rate signal. 2015.